3 Python 程序组织与调试

先修知识、学习目标与本章位置

本章以第 2 章的函数、列表、字典、条件与循环为基础，不要求软件工程经验。学习后应能把程序拆成职责明确的模块；辨认导入路径和数据路径；安全地读写文本、CSV 与 JSON；解释类、实例、方法、引用和复制；根据报错与最小测试定位错误；组织一次可从头复现的训练与评价。

本章侧重程序结构，预测模型仍采用第 1 章的均值基线和单近邻规则。第 4 章将把列表循环改写成数组计算，第 25 章与第 44 章再扩展到深度学习和较完整的工程项目。

一个计算 MAE 的函数可以只有十行，但一个完整实验还需要知道数据从哪里读取、怎样核对字段、哪个对象保存训练结果、结果写到哪里，以及失败时如何找到原因。若这些工作混在一个长脚本里，修改输入路径就可能影响训练，调整打印格式也可能意外改变计算。程序组织的目的，是使每一步的输入、输出和状态可以分别解释和检查。

本章先把“一个文件中的若干函数”扩展为“几个可以互相导入的文件”，再讨论对象与状态。后半章将这些机制放回机器学习流程：数据读取完成后才允许训练；验证选择结束后才使用最终测试标签计算指标；保存报告时同时记录数据标识和选择规则。数据均为人为构造的教学数据。

3.1 从脚本到模块：代码如何被复用

模块、名称空间与导入

模块（module）是可以被导入的代码组织单元。最常见的形式是一个`.py`文件，例如`metrics.py`。**名称空间**（namespace）保存名称与对象之间的对应关系；不同模块可以各自定义`mean`而不直接覆盖彼此。导入语句让当前模块获得访问另一个模块对象或其中某些名称的途径。

在同一目录保存`metrics.py`：

```
def mae(actual, predicted):
    if len(actual) != len(predicted) or len(actual) == 0:
        raise ValueError("expected equal, nonempty sequences")
    return sum(abs(p - y) for y, p in zip(actual, predicted)) / len(actual)
```

再保存`run.py`：

```
import metrics

score = metrics.mae([5, 5], [7, 3])
print(score)
```

运行`python3 run.py`得到 2.0。`metrics.mae`中的点表示先取得`metrics`所指模块，

再访问其中名为mae的对象。与第 2 章的list.append一样，点号用于属性访问；属性可以是函数，也可以是数据。

常见导入方式的区别如下：

语句	当前文件得到的名称	调用写法
`import metrics`	`metrics`	`metrics.mae(...)`
`import metrics as mt`	`mt`	`mt.mae(...)`
`from metrics import mae`	`mae`	`mae(...)`

别名只改变当前文件使用的名称，不复制模块，也不改变磁盘文件名。from metrics import mae并不会同时定义metrics这个名称。使用from metrics import *会把许多名称引入当前文件，使来源不清楚，本讲义不用这种形式。

导入标准库不需要另外安装。例如import math导入数学功能，from pathlib import Path导入路径类。第三方包则必须存在于当前解释器的环境中。import numpy失败时，应检查第 2 章的虚拟环境与依赖，不应把文件名改成numpy.py来“补齐模块”。

导入时会发生什么

导入不只是查看函数名称。通常第一次导入一个模块时，解释器会执行该模块的顶层语句，包括函数和类的定义、顶层赋值以及没有放入函数的打印和文件操作。def语句创建函数，但函数体等到调用时才执行。若在模块顶层直接写train_model()，导入也会启动训练。

在同一解释器进程中，已导入模块通常会被缓存，普通重复导入不会把全部顶层代码重新执行一次。修改文件后，交互环境可能仍持有旧模块；本书的初学实验采用重新启动脚本的方法，确保从当前文件重新开始，避免把“保存了新代码”误认为“正在执行新代码”。

定义：程序入口与主模块保护

直接运行的文件，其特殊名称__name__通常为"__main__"；被普通导入的文件，其__name__为模块名称。把运行流程放进main()，再以if __name__ == "__main__":保护，可以使文件同时支持直接运行和作为模块导入。

```
def main():
    print("experiment starts")

if __name__ == "__main__":
    main()
```

直接运行时条件成立，打印文字。被导入时函数仍然可用，但该分支不执行。双下划线名称是 Python 预定义机制的一部分，必须按原样拼写。入口保护只控制这一个分支，不会阻止分支之外的顶层语句执行。

例题 3.1：区分导入与直接运行

`helper.py`的内容如下。另一个文件只写`import helper`，会打印哪些内容？直接运行`helper.py`又会打印哪些内容？

```
print("loading")

def main():
    print("running")

if __name__ == "__main__":
    main()
```

解 新进程中首次导入时，第一行是顶层打印，因此输出`loading`；函数体没有被调用，不输出`running`。直接运行时先输出`loading`，随后入口条件成立，再输出`running`。若要求导入保持安静，应把第一行也移入`main`，而不是只在末尾添加保护语句。

查找路径、命名冲突与包的最小概念

模块查找路径是解释器寻找模块的位置集合，可通过`sys.path`查看。直接运行脚本时，脚本所在目录通常位于查找路径前部；标准库目录和当前环境中的第三方包目录也参与查找。它与后面打开数据文件时使用的当前工作目录不是同一个概念。

```
import sys
import metrics

print(sys.executable)
print(metrics.__file__)
```

两个`print`依次输出所用解释器的实际位置和当前导入模块的来源。若目录里有自建`random.py`、`json.py`或`numpy.py`，它可能遮蔽同名库。报错中出现“部分初始化模块”或预期属性不存在时，应核对模块来源，再给自建文件取不冲突的名称。

包（package）用于组织多个模块。例如一个目录中放置`__init__.py`，再包含`metrics.py`和`models.py`，就可以组成普通包。`import package_name.metrics`表示访问包内模块。Python 还支持其他包形式，本章不展开。`pip`安装的**分发包**是安装单位，安装名与导入名不保证相同；不能仅凭安装名猜出所有导入语句。

两个模块相互在顶层导入、又立即访问对方尚未建立的名称，可能形成**循环导入**。本章用单向依赖减少这一问题：入口导入数据、模型与指标模块，模型与指标不反向导入入口。共同的小函数可以放入独立模块，由需要的模块导入。

3.2 文件路径：数据位置与运行位置

绝对路径、相对路径与当前工作目录

绝对路径从文件系统的根位置确定文件，**相对路径**需要一个起点才能解释。`Path("data/parcels.csv")`通常相对于**当前工作目录**（current working directory，CWD），不是自动相对于写出这行代码的文件。编辑器、终端和其他程序启动同一脚本时，当前工作目录可能不同。

设项目位于`/tmp/demo`，入口为其中的`main.py`。从上层目录启动时执行：

```
python3 demo/main.py
```

此时相对数据路径从工作目录解释，两个位置分别是：

```
实际尝试读取：/tmp/data/parcels.csv
项目数据位置：/tmp/demo/data/parcels.csv
```

两者不同，所以项目中即使已有数据，也可能报找不到文件。

`pathlib`是标准库中的路径工具，`Path`对象表示路径并提供相关方法：

```
from pathlib import Path

print(Path.cwd())
base = Path(__file__).resolve().parent
data_path = base / "data" / "parcels.csv"
print(data_path)
print(data_path.is_file())
```

`__file__`在普通脚本中提供当前源文件路径；`resolve()`获得规范化的绝对位置并解析符号链接；`parent`取父目录。路径对象之间的`/`是拼接路径的运算，不是数值除法。`is_file()`检查当前位置是否为文件，但“刚才存在”不能保证稍后读取一定成功，实际打开仍需处理异常。

在交互解释器或某些笔记本中没有`__file__`，不能直接复制这一写法。本章配套实验是磁盘脚本，因而可以使用。符号链接会影响解析后的基准位置；实验未依赖符号链接，数据始终相对于真实脚本目录定位。

例题 3.2：同一脚本为什么换目录就失败

入口与数据的位置分别为：

```
入口：/tmp/demo/main.py
数据：/tmp/demo/data/a.csv
```

入口使用`open("data/a.csv")`读取。分别从项目目录与上层目录启动，解释结果，

并给出不依赖启动目录的写法。

解 从项目目录启动，相对路径指向实际数据；从`/tmp`启动，则指向`/tmp/data/a.csv`，若该文件不存在就失败。修正为先计算`base = Path(__file__).resolve().parent`，再用`base / "data" / "a.csv"`读取。导入能找到同目录模块，并不意味着数据相对路径也会采用该目录。

路径对象的常用操作

`path.name`取得含扩展名的文件名；`path.stem`取得去掉最后扩展名后的名称；`path.suffix`取得最后扩展名，例如`".csv"`；`path.parent`取得父目录。路径存在不等于内容格式正确，扩展名为 CSV 也不保证能读出期望字段。

输出目录需先建立：

```
output = base / "results"
output.mkdir(parents=True, exist_ok=True)
report_path = output / "summary.json"
```

`parents=True`允许创建缺失的上层目录；`exist_ok=True`允许目录已经存在，若同名对象是普通文件仍会失败。本章输出固定保存在项目的结果目录，重复运行会替换同名报告。需保留多组结果时，应使用明确的实验编号建立子目录，并记录每组配置，不能只依赖记忆辨认文件。

Windows 路径的分隔符与 macOS 不同，`Path`会按当前系统解释拼接。Python 字符串中的反斜线可能形成转义，如`"\n "`代表换行；Windows 手写路径可以使用原始字符串`r"C:\data \a .csv"`。本章优先采用`Path`拼接，目录含中文或空格不需要改名。

3.3 文本文件、编码与资源释放

文件对象与 with 语句

文件对象连接程序与打开的文件，记录读取或写入状态。`open`或`Path.open`建立连接后，操作系统资源需要释放。`with`使用**上下文管理器**（context manager）管理进入和退出时的操作；对普通文件，即使代码块抛出异常，也会在退出时关闭文件。

```
from pathlib import Path

path = Path("note.txt")
with path.open("w", encoding="utf-8") as stream:
    stream.write(" 重量\n 费用\n")

with path.open("r", encoding="utf-8") as stream:
    text = stream.read()
print(repr(text))
```

写模式`"w"`会在打开时截断已有文件，不能用它打开需要保留的原始数据。`"r"`读取；`"a"`在末尾追加，重复运行可能重复加入相同记录；`"x"`仅新建，已存在时失败。含

"b"的模式处理二进制字节，本节文本模式处理字符串。

编码（encoding）规定字符与字节之间的转换。显式使用 UTF-8 可减少不同机器默认编码的差异；用错误编码读取可能抛出`UnicodeDecodeError`，也可能得到乱码。不要用`errors="ignore"`无声丢弃字符来假装数据完好。文本中的`"\n "`表示换行，`write`不会自动补换行。

`repr(text)`返回适合观察转义和空白的字符串表示，因此上述输出可看到`' 重量\n 费用 \n '`。普通`print(text)`会实际换行。调试含空格的字段时，`repr`比直接打印更容易辨认空串和只含空白的串。

逐行读取、位置与大文件

`stream.read()`一次读入剩余全部文本，`readline()`读取下一行，`for line in stream`逐行遍历。文件对象有当前位置；一次`read()`读到末尾后，再次调用通常得到空串，不是文件内容突然消失。

```
with path.open("r", encoding="utf-8") as stream:
    for number, line in enumerate(stream, start=1):
        print(number, repr(line.rstrip("\n")))
```

这里`start=1`使显示行号从 1 开始；`rstrip("\n ")`去掉行尾换行，不等同于`strip()`去掉两端所有空白。是否允许去除空白取决于字段定义：成绩数字外的空格常可去除，文本内容中的空格却可能有意义。

若文件大小为 B 字节，一次完整读取通常需要与 B 同量级的内存；逐行迭代只需保留当前行及程序积累的结果。不过若每行都被追加到列表，最后仍会保存全部数据。逐行读取是一种访问方式，不自动等于低内存算法。

> **注意：保存成功与完整保存**
>
> `with`保证文件关闭，不保证一组多文件输出构成不可分割的整体。程序在写一半时终止，结果目录可能只含部分文件。本章把报告写入固定输出目录，适用于小型教学实验；严肃运行可先写临时文件，校验完成后替换目标，并用运行状态区分完成和失败。原始数据不应与输出使用同一路径。

3.4 CSV 与 JSON：给数据建立明确结构

CSV 的行、字段与表头

CSV（comma-separated values，逗号分隔值）是一类表格文本格式，通常每条记录一行、各字段用逗号分隔。字段本身可能包含逗号、引号或换行，需要引用和转义规则。因此不能把任意 CSV 都当成`line.split(",")`。

本章输入文件的开头为：

```
id,split,weight,cost
tr01,train,1.0,6.0
tr02,train,2.0,8.0
```

表头给出字段名称；id是记录标识；split说明所属集合；weight单位为千克；cost单位为元。id只用于追踪，不作为数值特征。读取 CSV 得到的"1.0"仍是文本，需显式转为数值。

```
import csv

with data_path.open("r", encoding="utf-8", newline="") as stream:
    reader = csv.DictReader(stream)
    print(reader.fieldnames)
    for row in reader:
        weight = float(row["weight"])
        print(row["id"], weight)
```

DictReader将每条记录映射为字典；reader.fieldnames给出表头。newline=""让 CSV 模块按自身规则处理换行，尤其避免某些系统上写入多余空行。文本转数值、范围检查、字段缺失检查仍是应用程序的责任。

输入验证：语法正确还不够

输入验证检查数据是否满足程序接口与任务要求。对本章 CSV，依次核对表头、字段数量、标识、集合名称和数值。三种集合都必须非空；标识在整个文件中唯一；重量和费用必须有限且非负。float("nan")和float("inf")可以转换成功，但前者表示非数值，后者表示无穷，均不符合本实验的测量值要求。

```
from math import isfinite

value = float("nan")
print(isfinite(value))
print(isfinite(float("inf")))
print(isfinite(0.0))
```

输出依次为False、False、True。只用value < 0不能排除 NaN，因为 NaN 参与通常的大小比较不符合普通实数规则。正确检查可以写isfinite(value) and value >= 0。

> **例题 3.3：一条可以转换的坏记录**
>
> CSV 中有a01,train,nan,6.0。程序只调用float，然后检查重量是否小于 0。为什么可能放过它？应在哪个步骤拒绝？

解 float("nan")返回特殊浮点值，转换没有抛异常；nan < 0为假，因此“若小于 0 就拒绝”的分支不执行。应在转换后验证有限性，并给出原记录位置。本章严格读取器拒绝整次实验，而不是悄悄删除记录，因为删除会改变已指定的划分和评价样本。

若数据任务明确允许丢弃坏行，也应保存拒绝原因、数量及原标识，并说明对结果的影响。第 2 章成绩清理采用“保留有效项并报告拒绝项”，本章项目采用“输入不合约定就停止”；两种策略适用于不同接口，不能在同一次运行中无说明地混用。

写出 CSV 使用`csv.writer`：

```
with report_path.with_suffix(".csv").open(
    "w", encoding="utf-8", newline=""
) as stream:
    writer = csv.writer(stream)
    writer.writerow(["id", "actual", "predicted"])
    writer.writerow(["te01", 6.8, 6.0])
```

`with_suffix`返回扩展名替换后的新路径，不修改原路径对象。`writerow`接收字段序列，负责 CSV 引用与分隔；`writerows`可以一次接收多条记录。

JSON 的结构与序列化

JSON（JavaScript Object Notation，JavaScript 对象表示法）是一种结构化文本格式，适合保存配置与汇总报告。常见对应关系为：Python 字典对应 JSON 对象，列表对应数组，字符串和数值对应相应值，`None`对应`null`，布尔值对应`true`或`false`。

序列化（serialization）把内存中的结构转换为可保存或传输的表示；**反序列化**恢复结构。JSON 文件保存的是数据，不会把 Python 类的方法一起保存下来。

```
import json

report = {"selected": "nearest", "test_mae": 0.85, "count": 4}
with report_path.open("w", encoding="utf-8") as stream:
    json.dump(report, stream, ensure_ascii=False, indent=2, allow_nan=False)
with report_path.open("r", encoding="utf-8") as stream:
    loaded = json.load(stream)
print(loaded["selected"])
```

`dump`写文件，`load`读文件；带字母`s`的`dumps`与`loads`分别生成字符串和解析字符串。`ensure_ascii=False`便于直接阅读中文，`indent=2`设置缩进，`allow_nan=False`拒绝 NaN 和无穷值，避免写出不符合标准 JSON 数值约定的报告。

JSON 对象的键最终是字符串；Python 元组保存后通常读回列表；路径对象和自定义模型不能默认直接写入 JSON，应先选取必要的字符串、数值、列表与字典。保存报告与保存可继续训练的模型是不同任务。第 25 章再讨论模型检查点。

3.5 异常处理：失败应当保留原因

异常对象与传播过程

异常(exception)表示执行中断正常流程的情况。第 2 章用`raise ValueError(...)`主动拒绝错误输入。本章进一步区分：函数负责发现什么错误，调用者决定能否恢复。若当前函数不捕获异常，它会向调用者传播；一直没有被处理时，程序终止并显示回溯（traceback）。

常见类型包括：`FileNotFoundError`表示找不到文件；`PermissionError`表示操作权限不足；`ValueError`表示对象类型通常可接受但取值不合适；`TypeError`表示操作

或参数类型不合适；KeyError表示字典缺少指定键；RuntimeError可用于不满足运行状态的情况。异常类型用于表达原因，不能只为了消除红色文字而随机更换。

```
try:
    value = float("missing")
except ValueError as exc:
    print("cannot parse:", exc)
else:
    print("parsed:", value)
finally:
    print("attempt finished")
```

try包含可能失败的操作；匹配的except处理异常；只有try正常完成才执行else；finally在正常或异常离开时执行，常用于清理。以上例子先打印无法转换的说明，再打印结束文字，不执行else。文件资源清理优先使用前面的with。

只捕获能够解释或处理的异常

若捕获转换失败后把值设为 0，损坏文本就变成合法测量，后续平均数看起来正常，却已改变数据含义。若使用except Exception: pass忽略所有普通异常，变量可能没有正确赋值，真正错误也失去位置。

更清楚的设计是把错误补充为可定位的信息，再继续抛出：

```
def parse_weight(text, row_number):
    try:
        value = float(text)
    except ValueError as exc:
        raise ValueError(f"row {row_number}: invalid weight") from exc
    if not isfinite(value) or value < 0:
        raise ValueError(f"row {row_number}: invalid weight range")
    return value
```

这里假定已导入isfinite，输入text为CSV 得到的字符串。raise ... from exc保留**异常链**，同时说明具体记录。float转换后的有限性检查放在try之外，使捕获范围与原因更明确。

入口可捕获预期的数据和文件错误，输出简短失败信息并以非零状态退出；调试时保留异常链有助于追踪。拼写错误产生的NameError等程序缺陷应直接暴露并修正，不应被统一解释为“用户数据有误”。

例题 3.4：异常处理也会制造逻辑错误

某程序对每个费用文本执行float，转换失败后except ValueError: value = 0.0，随后把所有值求平均。输入为["6", "missing", "10"]，它给出什么结果？为什么不应默认为正确均值？

解 程序实际计算 $(6 + 0 + 10)/3 = 16/3$。若任务规则为只统计两条有效记录，均值应为 8，且应报告一条拒绝记录；若规则为必须使用完整三条记录，则应停止并补充数

据。设为 0 相当于增加了一条零费用测量，没有数据依据。异常捕获能恢复控制流程，但不能替代任务中的缺失值规则。

3.6 对象、类与实例：保存状态并提供方法

对象不是面向对象语法才出现的事物

Python 中的整数、列表、函数、模块和路径都是**对象**（object）。每个对象具有类型、身份和相关数据或行为。`type(value)`检查类型，`id(value)`返回在对象存活期间用于标识其身份的整数，不用于保存跨运行的稳定编号。

类（class）描述一类对象的结构和行为；按类创建的具体对象称为**实例**（instance）。`Path("data/a.csv")`创建一个路径实例；`path.open()`调用其**方法**（method）。**属性**（attribute）是用点号访问的名称，既可能表示数据，也可能表示方法。理解这些术语后，后续库中的`model.fit(...)`就可以读成“调用模型实例的拟合方法”。

函数适合把输入映射为输出；类适合把一份需要持续保存的状态与操作它的方法放在一起。不是每个函数都需要改成类。MAE 只依赖给定的真实值和预测值，没有训练状态，作为普通函数很自然；模型则需要在`fit`以后保留从训练数据得到的内容。

一个完整的均值模型类

以下简化类假定`y`为非空有限数值列表，`x`与`y`已对齐；完整实验另加输入检查。

```
class MeanRegressor:
    def __init__(self):
        self.mean_ = None

    def fit(self, x, y):
        if len(x) != len(y) or len(y) == 0:
            raise ValueError("expected equal, nonempty data")
        self.mean_ = sum(y) / len(y)
        return self

    def predict(self, x):
        if self.mean_ is None:
            raise RuntimeError("fit must be called before predict")
        return [self.mean_ for _ in x]
```

`class`建立类；调用`MeanRegressor()`创建实例，随后`__init__`初始化实例属性。它是初始化方法，通常不返回实例，不能写`return self`。`self`是接收当前实例的形参名称，按约定如此命名，不是独立的关键字。

调用`model.fit(x, y)`时，Python 自动将`model`作为第一个参数传入，所以定义需要`self`，调用不再手动填写。`self.mean_`保存在该实例上，方法退出后仍可使用；局部变量`mean`若没有被保存或返回，则不能靠另一个方法直接读取。

下划线后缀`mean_`在这里用于提示“训练后得到的属性”，是命名约定，不是 Python

自动限制读写的机制。初始化为None表示尚未拟合，不能用if not self.mean_判断拟合状态，因为合法训练均值也可能为 0。

```
model = MeanRegressor()
returned = model.fit([1, 2, 3], [2, 2, 8])
print(returned is model)
print(model.mean_)
print(model.predict([0, 100]))
```

输出True、4.0 和[4.0, 4.0]。fit返回自身，支持连续调用：

```
model = MeanRegressor().fit(x, y)
```

这一写法假定已定义合法训练输入x与y。predict返回新列表，不修改输入。均值模型不使用输入数值来改变预测，但仍用x确定输出条数，并保持与其他模型相同的调用方式。

实例状态与共享的类属性

```
a = MeanRegressor().fit([1, 2], [6, 8])
b = MeanRegressor().fit([1, 2], [20, 30])
print(a.mean_, b.mean_)
```

两个实例分别保存 7.0 和 25.0。若把可变训练记录写成类体中的records = []，再通过实例向它追加，多个实例可能共用同一个列表。应该在__init__内部写self.records_ = []，使每个实例获得单独容器。

> **例题 3.5：一个模型对象还是两个模型对象**
>
> 令a = MeanRegressor()、b = a。先用a.fit([1], [6])拟合，再用b.fit([1], [10])拟合。此时a.predict([2])是什么？若需独立比较两套训练结果，应怎样创建对象？

解　b = a只增加一个指向同一实例的名称，因此第二次拟合把同一个mean_改为 10，预测为[10.0]。独立结果需要分别调用两次类，例如a = MeanRegressor()和b = MeanRegressor()。把同一个可变模型放入多个字典键，不会自动保存多个历史版本。

接口、组合与本章的面向对象边界

接口（interface）规定外部如何与对象交互，包括方法名称、参数、返回值、状态和错误。两个模型都提供fit(x, y)与predict(x)，入口就能用相同代码比较它们。接口相同不保证内部算法相同，也不保证训练结果可靠。

组合是让一个对象或流程使用其他对象完成工作，例如入口持有数据、模型和评价函数。继承允许以已有类为基础定义新类，本章无需依赖继承就能组织实验。后续 PyTorch 的nn.Module会使用继承，第 25 章再补充其必要语法。当前需掌握实例状态、方法调用和明确接口，不必先学习所有面向对象机制。

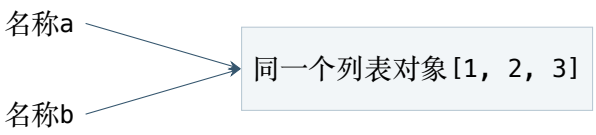
3.7 引用、可变对象与复制

名称绑定与原地修改

第 2 章已说明赋值绑定名称。**引用**（reference）可以理解为程序访问对象的联系；多个名称可以引用同一对象。执行`b = a`不意味着按字节复制`a`中的全部数据。

```
a = [1, 2]
b = a
b.append(3)
print(a, b)
print(a is b)
```

两个列表显示都为`[1, 2, 3]`，身份比较为`True`。`append`修改原对象；若改为`b = [9]`，则只把名称`b`重新绑定到新列表，`a`仍指向原列表。



可变对象（mutable object）允许改变对象内的内容，例如列表、字典和集合；**不可变对象**（immutable object）不能以这种方式改变自身内容，例如整数、字符串和元组。对整数执行`x += 1`会计算新整数并重新绑定名称，不是把所有引用原整数的名称一起改成新值。

`==`比较值是否相等；`is`比较是否为同一个对象。两个独立创建的`[1, 2]`通常值相等但身份不同。小整数或字符串的对象复用属于实现细节，不能用`is`代替数值或文本相等判断。对`None`使用`is None`则表达明确的身份检查。

函数传参为何可能影响调用者

函数形参同样绑定到实参对象。调用`change(values)`不会自动复制列表：

```
def rebind(values):
    values = [99]


def mutate(values):
    values.append(99)

items = [1, 2]
rebind(items)
print(items)
mutate(items)
print(items)
```

第一次输出`[1, 2]`，第二次输出`[1, 2, 99]`。`rebind`只改变局部名称，`mutate`通过引用修改了共享对象。与返回值无关：即使函数没有`return`，原地修改也可能已经

发生。

副作用（side effect）指函数除了返回结果外，对外部可观察状态产生的改变，包括修改传入列表、修改全局字典、写文件和打印。副作用并非都错误，`fit`更新模型状态就是有意设计的副作用；应明确哪些函数修改什么，不能把评价函数也写成会排序或删除输入记录的隐式修改。

浅复制与深复制

浅复制（shallow copy）创建新外层容器，但内部元素仍引用原来的对象。普通列表的`copy()`、`list(a)`和`a[:]`都可以建立浅复制。**深复制**（deep copy）递归复制内部结构，可用标准库`copy.deepcopy`。

```
import copy

rows = [[1, 6], [2, 8]]
shallow = rows.copy()
deep = copy.deepcopy(rows)
shallow[0][1] = 99
shallow.append([3, 10])
print(rows)
print(shallow)
print(deep)
```

输出依次为：

```
[[1, 99], [2, 8]]
[[1, 99], [2, 8], [3, 10]]
[[1, 6], [2, 8]]
```

新外层列表上的追加不影响原外层长度；共享的内层第一行被修改，则两边都能看到。

操作	外层身份	内层可变对象
`b = a`	相同	相同
`b = a.copy()`	不同	仍可能相同
`b = copy.deepcopy(a)`	不同	通常递归得到独立副本

深复制有时间和空间成本，对文件句柄、外部资源及特殊自定义对象也不能理解为“复制整个运行世界”。若本章训练记录是只含数值的元组，保存`tuple(zip(x, y))`已经可以隔离原输入列表的后续增删或数值替换，不需要随意深复制。

例题 3.6：二维列表的重复引用

预测下面代码的结果，并构造两个真正独立的内层列表。

```
rows = [[0, 0]] * 2
rows[0][0] = 7
print(rows)
```

解 列表重复运算复制的是其中的引用，两个位置都指向同一内层列表，输出`[[7, 0], [7, 0]]`。应写`rows = [[0, 0] for _ in range(2)]`，每次循环创建新内层列表。若每一行都用同一个外部列表`row`追加，即使使用循环也仍会共享；关键是何时创建对象。

默认参数与延迟读取的循环变量

第 2 章的可变默认参数问题可以用引用解释。`def collect(value, bucket=[])`在定义时只创建一次默认列表，后续未提供`bucket`的调用会引用它。用`None`作默认值，再在函数体中新建列表，才能实现“每次调用独立容器”。若调用者显式传入一个列表，函数仍会按接口修改它。

保存多个预测函数还可能遇到另一种问题：函数执行时才读取外层名称。

```
functions = []
for k in range(3):
    functions.append(lambda x: x + k)
print([f(10) for f in functions])
```

构建结束后`k`为 2，三个函数随后调用都读取此时的`k`，结果为`[12, 12, 12]`。要在每次创建函数时保存当前数值，可以使用默认参数：

```
functions = []
for k in range(3):
    functions.append(lambda x, k=k: x + k)
print([f(10) for f in functions])
```

结果为`[10, 11, 12]`。左侧`k`是函数形参，右侧`k`在创建时求值。本例保存不可变整数，含义明确；若保存的是可变列表默认值，还需重新检查共享关系。项目比较模型时直接创建不同实例，可以减少保存匿名函数时的这种绑定问题。

3.8 阅读回溯与检查中间状态

从最后一行判断失败类型

假设`run.py`调用`metrics.mae`，而函数内部遗漏了非空检查，可能看到如下回溯：

```
Traceback (most recent call last):
  File "run.py", line 4, in <module>
    print(mae([], []))
  File "metrics.py", line 3, in mae
    return total / len(actual)
ZeroDivisionError: division by zero
```

先读最后一行，确定是除以零；再读最近的调用位置，确认分母`len(actual)`；随后沿调用链向上追踪为什么传入空列表。修正可能位于指标函数的输入检查，也可能

位于前面把数据筛空的代码，应根据接口决定。直接把分母改成`len(actual) + 1`会改变指标定义，不能算修复。

回溯所标行号是实际保存文件中的位置。编辑器里未保存的修改尚未执行；运行了同名旧副本时，也可能出现“修改过却没有变化”。应结合`__file__`、解释器位置和实际命令确认正在运行哪份程序。

有问题意识地打印

调试输出应针对一个假设。若怀疑 CSV 数值没有转换，应检查字段值的类型；若怀疑划分为空，打印每组长度；若怀疑特征与标签错位，打印带标识的前几条对应记录；若怀疑共享列表，打印`a is b`和内部对象的身份关系。

```
print("source:", data_path)
print("counts:", len(train), len(validation), len(test))
print("first train:", train[:2])
print("label type:", type(train[0]["cost"]).__name__)
```

这一片段需在相应变量建立后执行。先检查`train`非空，再访问`train[0]`。只打印最终 MAE 无法分辨数据读取、模型拟合和指标计算分别是否正确。

断点（breakpoint）让程序暂停在指定位置，以便观察变量并逐步执行。Python 内置`breakpoint()`通常进入交互调试器；常见操作包括查看变量、单步执行和继续运行。编辑器也可提供图形断点。对本书小程序，先用小输入、状态表和少量打印足以定位大多数问题；避免一次打印整份大数据掩盖关键记录。

最小可复现示例

最小可复现示例（minimal reproducible example）只保留复现同一个错误所需的输入和步骤。若完整项目报告错误，不必一开始就重读全部数据。可以先把触发记录保存成三行 CSV，直接调用读取器；若读取正常，再用两条训练记录和一个查询调用模型；最后单独计算一个手算可核对的指标。

缩小问题时必须保留触发条件。广播错误需要保持相关形状，引用错误需要保留共享对象，编码问题需要保留原字符。把所有数据换成简单数字后不再出错，并不证明原程序正确，只说明已删掉某个触发因素。

例题 3.7：定位“模型总预测同一个数”

一个程序的所有预测都是 9。给出至少三个可能原因，并分别设计能区分原因的检查。

解 第一种可能是程序确实选中了训练均值模型，检查对象类型、选择记录和训练均值即可。第二种可能是近邻函数在循环外固定使用第一个查询，应输入相距很远的两个查询，并打印每次距离。第三种可能是保存多个候选时反复引用同一个实例，检查候选对象身份及每次拟合后的状态。还可能是输入列被错误读成常数，应按标识打印原字段和转换后的值。常数输出本身不能唯一证明模型没训练。

3.9 断言、最小测试与可靠核对

assert 的用途与限制

断言（assertion）写出开发者预期始终成立的内部条件。例如检查输出条数：

```
assert len(predicted) == len(test)
```

条件为假时会抛出AssertionError。断言适合教学核对与测试，但在启用优化模式的 Python 运行中可能被移除。对外部输入的必要检查仍应使用显式条件判断与raise，不能只依赖断言。

```
from math import isclose
from metrics import mae

assert isclose(mae([5, 5], [7, 3]), 2.0, rel_tol=0, abs_tol=1e-12)
assert mae([0], [0]) == 0
```

isclose使用浮点容差。第一个测试包含一正一负有符号误差，能揭示遗漏绝对值的错误；只用[1, 2]与[2, 3]，错误的有符号平均也会偶然得到 1。核对输入应针对要识别的错误，而非只追求测试数量。

正常、边界和失败路径

对 MAE，应包含等长正常输入、全零误差、空输入、长度不等和非有限数。对模型，应包含预测前未拟合、两次独立实例、距离同分和输入后来被修改。对文件，应包含错误表头、缺字段、多字段、重复标识和未知集合。每项检查都对应一个明确承诺。

捕获预期异常的最小写法为：

```
try:
    mae([], [])
except ValueError:
    pass
else:
    raise AssertionError("empty inputs were not rejected")
```

这里的pass表示测试已经收到所要求的异常，不是项目运行时忽略失败。若没有报错，else让测试失败；若报了其他类型的异常，也应暴露出来，因为那并不符合约定。

单元测试（unit test）核对一个较小组件；**集成测试**（integration test）核对多个组件连接后的行为。分别检查读取器和模型，不能替代检查整条流程是否仍将正确标签交给指标。反过来，只核对最终 MAE 也可能让两个错误彼此抵消。完整项目保留少量两类检查。

使用标准库 unittest

unittest是标准库测试工具，以方法组织测试，并报告哪些检查失败。本章test_project.py包含以下形式：

```
import unittest
from metrics import mae

class MetricTests(unittest.TestCase):
    def test_hand_value(self):
        self.assertAlmostEqual(mae([5, 5], [7, 3]), 2.0)

    def test_empty(self):
        with self.assertRaises(ValueError):
            mae([], [])

if __name__ == "__main__":
    unittest.main()
```

`TestCase`提供核对方法，`test_`开头的方法被识别为测试。`assertAlmostEqual`检查浮点结果，`assertRaises`检查是否抛出指定异常。此处继承仅用于沿用测试工具已定义的功能，无需先设计复杂类层次。命令`python3 test_project.py`会运行测试并打印通过数量。

测试不能证明所有可能输入都正确，但可以固定已经明确的规则，帮助修改代码后发现退步。若把预期值写成与被测函数完全相同的计算，就容易共同犯错；本章优先采用手算小例、独立实现对照和特定失败输入。

3.10 实验 3.1：对象变化与文件往返

入口为配套实验`/ch03_objects.py`，仅依赖 Python 标准库。程序不要求输入个人文件，文件实验在自动建立的临时目录中进行，结束后清理。实验同时核对三个问题：哪些修改会穿过共享引用；怎样在每次调用时创建独立列表；JSON 写入再读回是否保持预期数据。

开始前手工画出`rows`、`alias`、`shallow`、`deep`指向的外层和内层对象。核心操作为：

```
rows = [[1, 6], [2, 8]]
alias = rows
shallow = rows.copy()
deep = copy.deepcopy(rows)
alias.append([3, 10])
shallow[0][1] = 99
```

`alias.append`修改原外层列表，所以`rows`增加一行，`shallow`的长度仍为 2。第二次操作修改共享内层第一行，所以`rows`和`shallow`的费用都变为 99。`deep`保存的原始两行不变。

在项目的配套实验目录运行：

```
python3 ch03_objects.py
```

主要预期输出为：

```
rows: [[1, 99], [2, 8], [3, 10]]
shallow: [[1, 99], [2, 8]]
deep: [[1, 6], [2, 8]]
identity: True False True False
fresh: [1] [2]
captured: [10, 11, 12]
round trip: 3 教学数据
caught: ValueError
all checks passed
```

身份比较依次核对别名与原列表、浅复制与原列表、浅复制第一行与原第一行、深复制第一行与原第一行。结果应精确一致；本实验没有近似数值计算。

文件往返部分使用`TemporaryDirectory`作为上下文管理器。进入时创建临时目录，将字典写入`report.json`再读回；退出时删除本实验临时目录。这是标准库管理临时资源的实例，不能将它替换为随意删除原始数据目录。

修改任务依次进行，每次从原版恢复：把`shallow`改为深复制，预测第一行还会不会互相影响；把默认参数改成`bucket=[]`，连续调用观察累积；去掉`lambda`中的`k=k`，预测三个输出；把 JSON 中的整数改成元组`(1, 2)`，读回后核对类型。最后一个任务应得到列表，说明数据往返不保证保留所有 Python 类型。

3.11 实验 3.2：组织可复现的预测小程序

目录、输入与流程约定

入口为配套实验`/ch03_project/main.py`。**可复现**（reproducible）在本章指：给定同一代码、数据、环境和流程规则，可以从头得到一致的预测和评价结果。它不等同于模型对未来数据一定可靠，也不要求不同机器生成完全相同的耗时或临时路径。

```
ch03_project/
    main.py
    io_utils.py
    models.py
    metrics.py
    test_project.py
    data/parcels.csv
    results/summary.json
    results/predictions.csv
```

`main.py`协调流程；`io_utils.py`负责严格读取与结果写出；`models.py`保存模型类；`metrics.py`提供 MAE；`test_project.py`核对规则。结果文件由运行生成。依赖关系由入口指向各组件，组件不导入入口，导入模块不启动实验。

数据有 12 条，训练、验证、测试各 4 条，划分直接由 CSV 的`split`列指定。本章不重新随机划分，因此没有用一个随机种子代替数据记录的必要。数据如下，重量和

费用均为教学构造值。

集合	重量（千克）	费用（元）
训练	1，2，3，4	6，8，10，12
验证	1.2，2.2，3.2，3.8	6.4，8.4，10.4，11.6
测试	1.4，2.6，3.4，4.5	6.8，9.2，10.8，13.0

候选为训练均值模型和单近邻模型；按最小验证 MAE 选择，同分时取声明顺序靠前者。单近邻距离同分时取训练记录中靠前者。选定后不把验证集合并重训，直接评价已选模型；这样能明确追踪每次拟合到底使用哪些标签。其他项目可以另定重训协议，但必须记录并保持一致。

严格读取与保持记录对齐

读取器先验证`reader.fieldnames`等于预期表头。每条记录转换前检查字段数、非空唯一标识和合法集合名。完整读取器用`reader.line_num`记录当前记录结束的物理行号。CSV 字段可以含引用的换行，因此记录序号不总等于文件行号。对`DictReader`，多出的字段可能对应键`None`，缺少的字段可能对应值`None`；因此完整读取器有如下检查：

```
if None in row or any(v is None for v in row.values()):
    raise ValueError(f"row {line}: wrong field count")
```

通过检查后再解析`float`，验证有限、非负，最后建立规范字典。这一过程先保证数据结构，再保证字段意义，不能只看能否运行除法。完整实现保存在`io_utils.py`；讲义中的短片段用于说明该检查的原因，运行时使用完整入口。

训练输入和标签必须从同一记录顺序提取：

```
def columns(records):
    return ([row["weight"] for row in records],
            [row["cost"] for row in records])
```

这里两次遍历同一个未被修改的记录列表，因此位置对应。更换顺序或筛选时，应对整条记录操作，而非分别排序重量和费用。唯一标识只能揭示某些重叠，不能证明不同标识没有复制内容；真实数据仍需按第 1 章协议检查来源与重复样本。

用相同接口比较两个模型

均值模型的`fit`保存`sum(y) / len(y)`。单近邻模型的`fit`保存`tuple(zip(x, y))`，`predict`逐个查询最近训练记录：

```
return [min(self.records_, key=lambda row: abs(row[0] - q))[1]
        for q in x]
```

这一片段展示方法内部的核心选择规则，假定已完成拟合与输入检查，且差值处

于可表示范围。完整模块还会检查均值、距离和指标结果是否有限；输入各项有限，并不保证求和或相减后仍有限。若范围检查失败，程序停止并要求检查数据尺度，第 4 章将进一步讨论数值异常。`min`在同分时选第一次遇到的记录；`[1]`取得费用。与第 1 章完整排序相比，单个近邻只需扫描找到最小值，无需把所有距离排序。对 n 条训练记录、m 个查询，时间为 $O(mn)$；保存训练记录需要 $O(n)$ 空间，保存输出需要 $O(m)$。这里 O 表示随输入规模增长的量级，不表示实际秒数。

入口中每个候选都创建新实例：

```
factories = {"mean": MeanRegressor, "nearest": NearestRegressor}
models = {}
scores = {}
for name, factory in factories.items():
    model = factory().fit(x_train, y_train)
    models[name] = model
    scores[name] = mae(y_val, model.predict(x_val))
selected = min(scores, key=scores.get)
```

`factory`在这里是类对象，`factory()`创建对应实例。两个字典分别保存模型和验证分数，不把模型对象与浮点分数混在一起。`scores.get`作为比较函数，让`min`按键对应的分数选择；字典的插入顺序对应已声明的候选顺序。

读取器会一次读入文件全部字段，但候选拟合只接收训练标签，候选选择只接收验证指标。文件中存有测试标签不自动构成泄漏，关键是这些标签是否影响训练或选择；本章按代码依赖限制其用途。真实项目也可把最终测试标签交给独立评价程序管理。

手算验证选择与最终评价

训练均值为 $(6+8+10+12)/4 = 9$。在验证集，四个绝对误差为 2.6、0.6、1.4、2.6，因此 MAE 为 1.8。单近邻分别找到重量 1、2、3、4，预测 6、8、10、12，绝对误差均为 0.4，MAE 为 0.4，应选择单近邻。

测试只对选定模型作最终报告，同时保留预先声明的均值基线作比较：

重量	真实费用	单近邻预测	单近邻绝对误差	基线绝对误差
1.4	6.8	6	0.8	2.2
2.6	9.2	10	0.8	0.2
3.4	10.8	10	0.8	1.8
4.5	13.0	12	1.0	4.0

单近邻测试 MAE 为 $(0.8 + 0.8 + 0.8 + 1.0)/4 = 0.85$；均值基线为 $8.2/4 = 2.05$。4.5 超过训练重量范围，单近邻仍返回边界费用 12；这体现近邻方法的外推限制。该数据构造得十分规则，结果不能用于推断真实物流业务表现。

运行、保存与再次运行

在配套实验目录执行：

```
python3 ch03_project/main.py
python3 ch03_project/test_project.py
```

也可从其他目录用入口的绝对路径运行。程序应输出：

```
counts: {'train': 4, 'validation': 4, 'test': 4}
validation mean: 1.800000
validation nearest: 0.400000
selected: nearest
test predictions: [6.0, 10.0, 10.0, 12.0]
test MAE: 0.850000
test baseline MAE: 2.050000
saved: results/summary.json, results/predictions.csv
```

第二个命令应报告 5 项测试通过。报告数字按六位小数显示，内部计算允许约 10^{-12} 的绝对误差。每条预测、集合计数和被选名称应一致，不把打印舍入差异解释为模型改变。

JSON 记录 Python 版本、数据文件的 SHA-256 摘要、集合数量、候选顺序、选择规则、是否重训、验证分数和最终指标。**摘要**（hash digest）把文件内容映射为定长标识，有助于判断两次运行是否使用相同字节；它不证明数据真实、标签正确或划分合理。仅修改 CSV 换行也会改变摘要，但模型数值可能仍相同。

预测 CSV 保存标识、真实值、预测值与逐条绝对误差。再次运行会替换同名结果；代码与数据不变时，关键数值应相同。跨 Python 版本的版本字段自然不同，JSON 字节完全相同不是本章复现要求。需要保留多次结果时，应复制到明确编号目录，并同时保留对应代码与输入。

有控制地修改与故障注入

从原始数据副本开始，每次只改一个因素，运行后恢复原版。

1. 把某重量改为`nan`，应在读取阶段失败并提示记录位置，不生成新的完整成功结果；旧输出若仍存在，也不能当作本次成功证据。
2. 把一条验证记录的标识改成训练标识，应拒绝重复。将集合名称拼成`validaton`，应拒绝未知集合。
3. 从其他工作目录启动入口，数值应与原版一致；若改成裸相对数据路径，应能复现路径错误，再恢复正确定位。
4. 将第一条训练费用 6 改为 10，先手算两候选验证分数，再运行。不能根据测试分数把该修改说成真实改进。
5. 单独对重量 100 作预测，应得到 12；该输入没有真实标签时，不并入 MAE。解释为什么程序能给出数值并不保证外推可靠。

第 4 项的均值变为 10，验证 MAE 为 1.8；单近邻验证误差为 3.6、0.4、0.4、0.4，MAE 为 1.2，仍选单近邻。修改后最终测试误差变成 3.2、0.8、0.8、1.0，MAE 为 1.45。该手算用于核对流程传播，不能用最终测试数值倒推应怎样修改训练数据。

3.12 知识联系与自检

程序中的三类联系需要分别理解。模块依赖说明代码从哪里取得；对象引用说明哪些名称共享状态；数据依赖说明哪些记录影响模型和选择。正确的导入结构不能自动防止标签泄漏，独立的对象也不能自动防止读错文件。

第 2 章函数的输入输出约定在本章扩展为模块接口和模型方法；列表切片的浅复制与本章的内部共享相连；第 1 章的训练、验证、测试在入口中表现为不同的数据传递。第 4 章增加数组以后，除这些要求外，还必须核对轴、形状和共享内存。

自检时应能回答：为什么脚本可导入却读不到数据；为什么`b = a`不是模型快照；为什么捕获错误后填 0 会改变任务；为什么空输入应由显式检查拒绝；为什么一个测试应包含正负有符号误差；为什么相同种子不能代替保存数据版本。回答应举出具体语句或输入，而非只复述术语。

3.13 分层习题

以下编号为 3.1–3.14。代码题先写预期状态，再运行核对；文件题使用自己建立的临时数据，不修改原始教学数据。

3.1 **概念**。区分脚本、模块、包和第三方依赖；说明`from metrics import mae`后为什么不能直接推断名称`metrics`可用。

3.2 **执行分析**。在例题 3.1 中将第一行打印移入`main`，直接运行和首次导入各输出什么？重复导入为什么通常不再次执行顶层语句？

3.3 **路径**。工作目录为`/tmp`，源文件在项目目录中。对以下两种定位写出实际指向的位置。

```
源文件：/tmp/project/main.py
路径甲：Path("data/a.csv")
路径乙：Path(__file__).resolve().parent / "data" / "a.csv"
```

3.4 **文件**。解释`"r"`、`"w"`、`"a"`的区别，为什么打开原始数据时不能误用`"w"`？`with`解决什么问题，不能保证什么？

3.5 **输入验证**。实现`parse_nonnegative(text)`，接受可转为有限非负浮点数的文本，其他情况抛`ValueError`。核对`"0"`、`"nan"`、`"inf"`、`"-1"`与`"missing"`。

3.6 **引用**。假定已导入`copy`，执行以下语句后写出四个变量的值。

```
a = [[1, 2], [3, 4]]
b = a
c = a.copy()
d = copy.deepcopy(a)
b.append([5, 6])
c[0][0] = 9
```

3.7 **副作用**。比较函数内部`values = sorted(values)`与`values.sort()`对调用者列表的影响。设计一个输入和检查区分两者。

3.8 **类与状态**。为什么均值模型未拟合标记用`None`，而不使用 0？若`a`和`b`引用同一模型，连续拟合会有什么后果？给出独立创建的写法。

3.9 **绑定**。三个函数在循环中通过`lambda x: x + k`保存，循环结束再调用得到什么？改为保存当前整数`k`，并解释默认参数求值时机。

3.10 **异常与测试**。有人用`except Exception: return 0`包装整个 MAE 函数，称“任何输入都不报错”。指出这怎样破坏接口，并写一个检查空输入必须报错的最小测试。

3.11 **项目手算**。实验 3.2 将第一条训练费用改为 10，其他不变。计算两候选验证 MAE、选定模型和测试 MAE，说明哪些结果必须重新生成。

3.12 **JSON**。字典`{"pair": (1, 2), "missing": None}`经过 JSON 写入与读回后，各值是什么类型？为何不应直接保存模型实例？

3.13 **故障定位**。项目报`KeyError: 'cost'`。给出应依次检查的三个具体位置；为什么不应直接改成`row.get("cost", 0)`？

3.14 **综合实验**。从项目目录与另一目录分别运行实验 3.2，再运行其测试。核对预测文件、数据摘要和指标；列出报告中哪些字段用于复现、哪些条件仍不足以证明真实泛化。

3.14 习题参考答案

3.1–3.4：组织、路径与文件

3.1 脚本强调作为入口执行的文件；模块强调可导入的代码单元，同一文件可以兼具两种用途；包组织多个模块；第三方依赖是需要安装到解释器环境中的外部代码。`from metrics import mae`只把指定名称绑定到当前名称空间，不同时绑定`metrics`；需用`import metrics`才能按模块前缀访问。

3.2 移入后直接运行输出`loading`与`running`，首次导入不输出文字。普通重复导入通常复用进程中已缓存模块；重启解释器后会重新建立模块。入口保护控制是否调用`main`，缓存则控制普通重复导入是否重新初始化，两个机制不同。

3.3 前者按当前工作目录解释为`/tmp/data/a.csv`；后者以源文件父目录为起点，指向`/tmp/project/data/a.csv`。`resolve`解析符号链接，本题假定没有符号链接影响。两者不能仅从路径字符串都含`data`就视为相同。

3.4 `"r"`读取且文件应存在；`"w"`写入并在打开时截断原内容；`"a"`在末尾追加，重

复运行可能重复记录。with保证按上下文管理规则退出，普通文件会被关闭；不能保证写入内容符合任务，也不能保证多个文件在故障时全部完整写入。

3.5–3.9：验证、引用与对象

3.5 先转换，再检查有限与非负；不能以真假判断代替非负检查。

```
import math

def parse_nonnegative(text):
    try:
        value = float(text)
    except ValueError as exc:
        raise ValueError("not a number") from exc
    if not math.isfinite(value) or value < 0:
        raise ValueError("expected finite nonnegative number")
    return value
```

"0"返回 0.0，其余四项抛ValueError。本题接口接受文本，不承诺任意 Python 对象都被统一转为同一错误。配套ch03_solutions.py核对这些输入。

3.6 四个名称的值为：

```
a, b: [[9, 2], [3, 4], [5, 6]]
c:    [[9, 2], [3, 4]]
d:    [[1, 2], [3, 4]]
```

追加通过共享外层作用于a和b；第一项修改通过共享内层作用于a、b、c；d在两次操作前已递归复制。

3.7 函数内values = sorted(values)建立新排序列表并重绑局部名称，调用者列表不变；values.sort()原地排序，调用者看到变化。以items = [3, 1, 2]分别传入，调用后核对原列表与[3, 1, 2]是否相等即可。若原列表本来已排序，这个输入就无法揭示两者行为差异。

3.8 均值 0 是合法训练结果，真假判断会把它误当成未拟合。None明确标记缺少训练状态。同一模型被两个名称引用时，后一次拟合覆盖前次状态；独立写a = MeanRegressor()与b = MeanRegressor()，再分别拟合。若用字典保存比较结果，也应每次创建新实例。

3.9 循环结束k=2，原版三个函数对 10 都返回 12。改成lambda x, k=k: x + k后，创建时分别保存 0、1、2，对 10 返回 10、11、12。这里默认值在函数对象创建时求值，不是在后续每次调用时重新读取循环变量。

3.10–3.14：失败路径与复现

3.10 返回 0 把输入错误、程序缺陷和真正零误差合并为一个无法区分的结果，可能让错误方案看起来最好。可使用本章的try/except/else最小测试，或在unittest.TestCase方法内写：

```
with self.assertRaises(ValueError):
    mae([], [])
```

测试中捕获预期异常是核对失败路径；实际评价中忽略异常并给出零分则改变接口含义。

3.11 训练费用变为 10、8、10、12，均值 10。验证均值误差 3.6、1.6、0.4、1.6，总和 7.2，MAE 为 1.8。单近邻验证误差 3.6、0.4、0.4、0.4，总和 4.8，MAE 为 1.2，选单近邻。测试预测为 10、10、10、12，误差 3.2、0.8、0.8、1.0，MAE 为 1.45。数据摘要、验证分数、选择报告和逐条预测应一起更新。用测试分数选择是否保留该修改会违反原协议。

3.12 `pair`读回为列表，`missing`读回为`None`。JSON 保留常用数据结构，不保存任意类的方法、运行资源或全部类型身份。模型实例应选取必要配置与训练状态按明确格式保存；本章只保存结果报告，不承诺可从报告恢复模型。

3.13 先检查原 CSV 表头是否真有`cost`，包括两端空格与大小写；再检查读取器构造字典时是否换名或漏字段；最后检查报错行变量是否仍是预期记录，而不是另一种字典。默认 0 会把缺字段变成测量值，掩盖格式错误，并可能改变训练目标或评价标签。应在读取边界拒绝不合约定的数据。

3.14 两种启动位置应给出相同集合数量、模型选择、预测与关键指标；数据未变时摘要一致。记录 Python 版本、代码版本或副本、数据内容、划分、候选顺序和同分规则，才能解释复现条件。本项目还需保留主源及配套代码的对应版本。即使这些条件满足，也只能复现教学实验，不能保证未来分布、标签质量和真实业务代价与该数据相同。

3.15 本章小结

模块给代码划分名称和依赖边界，路径给文件确定位置，数据格式与验证确定输入含义。类把训练状态和预测方法连接起来；引用决定状态是否共享，复制决定哪些层次独立。调试通过回溯、中间状态和最小输入寻找原因，测试把已明确的规则固定为可重复检查。

实验 3.2 将这些机制落实为可从头运行的预测流程。结果是否可复现，要检查代码、数据、环境和选择协议；评价是否可信，还要回到第 1 章对信息可用性、划分和泛化条件的要求。

4 NumPy 与向量化计算

先修知识、学习目标与本章位置

本章需要第 2 章的循环、列表和函数，以及第 3 章的导入、对象、引用与调试。矩阵运算只用到按行列取值、乘积求和和平均数，所需含义在本章补充，第 6 章再系统复习线性代数。

学习后应能解释数组的类型、轴和形状；区分索引、切片、视图与复制；逐轴核对广播和矩阵乘法；用归约与向量化替代逐样本循环；控制随机数并检查浮点结果；从头完成一个无预处理泄漏的数值实验。

第 2 章用循环逐条计算预测、有符号误差与平均误差，这种写法直接反映计算过程。数据量增加以后，可以把规则相同的操作交给专门的数值库一次完成。**NumPy**（Numerical Python）是 Python 科学计算中的基础数组库，提供多维数组及其运算。NumPy 数组也会遇到逻辑错误：程序能运算，只说明类型与形状在规则上兼容，不说明每一行仍对应正确样本。

本章把数组理解为带有形状与数据类型的数值容器。先逐项说明它与列表的不同，再建立逐元素运算、归约、广播和矩阵乘法的联系。所有维度推导都回到元素下标，避免只凭代码短或输出形状合理就判断正确。

4.1 安装、导入与数组的用途

让解释器与依赖处于同一环境

NumPy 是第三方依赖，需安装到实际运行脚本的 Python 环境中。按第 2 章建立并激活虚拟环境后，可在普通终端执行：

```
python3 -m pip install numpy
python3 -c "import numpy as np; print(np.__version__)"
```

Windows 按第 2 章将`python3`换为相应启动命令。`-m pip`让所选解释器运行包安装工具，减少把包装到另一个解释器的情况。若已安装且能导入，无需重复安装。网络不可用时可使用已经提供 NumPy 的环境，本书实验不依赖下载数据。

代码中采用惯用别名：

```
import numpy as np

values = np.array([1.0, 2.0, 3.0])
print(values)
print(type(values))
```

输出数值数组和其类型`numpy.ndarray`。`np`只是`numpy`的当前名称，不是 Python 内置关键字；`ndarray`表示多维数组类型。后文代码框未重复导入时，均以此前已执行`import numpy as np`为前提。

配套实验在 Python 3.12 与 NumPy 2.3.5 环境实际核对。基础接口不依赖特定硬件；

复现时应记录自己的版本，而不是只写“已安装 Python”。安装命令不固定后续所有版本行为，项目要长期复现时应同时保留已核对的依赖版本。

列表运算与数组运算不同

```
items = [1, 2, 3]
array = np.array(items)
print(items * 2)
print(array * 2)
print(items + items)
print(array + array)
```

列表乘 2 得到`[1, 2, 3, 1, 2, 3]`，表示序列重复；数组乘 2 得到`[2 4 6]`，表示每个元素乘 2。列表相加连接序列，数组相加执行逐位置加法。NumPy 显示数组时通常不在数字间添加逗号，显示格式不意味着它变成另一种数学对象。

向量化（vectorization）在本章指把逐元素或逐样本的规则表达为数组级操作，使主要循环由数值库执行。它没有消除底层循环，而是减少 Python 层逐项调度，并利用面向数组的实现。列表推导式虽然简短，仍然主要是 Python 逐项执行，不等于 NumPy 向量化。

NumPy 不自动把所有工作放到 **GPU**（graphics processing unit，图形处理器）上，本章标准数组在 **CPU**（central processing unit，中央处理器）上计算。深度学习中的设备与张量将在第 25 章讲解；数组代码正确与使用哪种硬件是两个问题。

4.2 数组形状、轴与统一的数据约定

shape、ndim、size 和 dtype

形状（shape）给出每个**轴**（axis）的长度；**维数**在数组语境中常指轴的个数，用`ndim`表示；`size`表示总元素数量；`dtype`表示元素数据类型。四者承担不同工作。

```
X = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
print(X.shape)
print(X.ndim)
print(X.size)
print(X.dtype)
```

分别得到`(2, 3)`、2、6 与`float64`。`shape`是元组；`(3,)`表示单轴长度 3，逗号不可省略；零维数组形状为`()`。若形状为 $(s_0, \dots, s_{r-1})$，总元素数为各长度的乘积，轴数为 r。对矩阵形状 (n, d)，`len(X)`只给出第一轴长度 n，不等于总元素数 nd。

创建方式	形状	结构含义
`np.array(3.0)`	`()`	零维数组，含一个数
`np.array([1, 2, 3])`	`(3,)`	一维数组，没有独立行轴与列轴
`np.array([[1, 2, 3]])`	`(1, 3)`	二维数组，一行三列
`np.array([[1], [2], [3]])`	`(3, 1)`	二维数组，三行一列

同样三个数，可以有不同形状。数学中的列向量常写成 3×1，NumPy 中的长度 3 一维数组却不自动带有“列”方向。`w.T`对一维数组仍为一维，必须明确增加轴才能得到二维列数组。

样本轴与特征轴

本书对普通表格数值数据采用

$$X \in \mathbb{R}^{n \times d}, \qquad X_{ij} = \text{第 } i \text{ 个样本的第 } j \text{ 个特征}.$$

数学下标在推导中通常从 1 开始，Python 索引从 0 开始，程序位置`X[0, 0]`对应数学 X_{11}。n 为样本数，d 为每个样本的特征数，`X.shape == (n, d)`。轴 0 沿样本变化，轴 1 沿特征变化。

例如两件包裹分别有三个已数值化特征，数组可以为

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

第一行是一个样本，不是一个特征的全部测量。若把矩阵转置为 3×2，数值总数不变，但样本与特征的解释互换。形状必须与语义一起写，不能只写“这是一个二维数组”。

> **注意：数组维数与特征维数**
>
> 100 个样本、每个样本有 20 个特征，可表示为形状`(100, 20)`的二维数组；特征空间则是 20 维。说“数据是 20 维”与说“数组有两个轴”并不矛盾，它们描述不同事物。后续一批图像可能有样本、高度、宽度、通道等轴，仍需逐轴解释。

reshape、增加轴与转置

`reshape`按指定顺序重新组织元素，要求总元素数量不变：

```
a = np.arange(6)
print(a.reshape(2, 3))
print(a.reshape(3, 2))
print(a.reshape(-1, 1).shape)
```

`arange(6)`给出 0 至 5；默认按行顺序填入。前两次分别得到两行三列和三行两列；`-1`表示由其他轴长度推算该轴，最后形状为`(6, 1)`，最多只能有一个待推算轴。没有依据地`reshape`可能让一个样本的特征散落到多行，即使元素数正确仍破坏数据含义。

增加长度为 1 的轴可用None或np.newaxis：

```
w = np.array([1.0, 2.0, 3.0])
print(w[:, None].shape)
print(w[None, :].shape)
print(w.T.shape)
```

依次为(3, 1)、(1, 3)和(3,)。二维数组X.T交换两个轴；reshape(3, 2)与X.T一般不是同一运算。对前面的 X，转置为 $\left(\begin{smallmatrix}1&4\\2&5\\3&6\end{smallmatrix}\right)$，而直接重排为 $\left(\begin{smallmatrix}1&2\\3&4\\5&6\end{smallmatrix}\right)$。

reshape可能返回共享数据的视图，也可能因为内存布局而复制，不能一律当成独立副本。需要独立数据时显式copy()。squeeze可去除长度为 1 的轴，但在单样本批次中不加选择地挤压，可能把样本轴也去掉；优先明确指定要处理的轴。

4.3 数据类型、数组创建与数值范围

同一 dtype 与固定宽度

普通数值数组的元素共用一个dtype。float64通常以 64 位表示浮点数，float32通常以 32 位表示；int64表示固定宽度整数；bool表示布尔值。位数影响可表示范围、精度和内存，不表示小数点后固定保存 64 或 32 位。

```
counts = np.array([1, 2, 3], dtype=np.int64)
weights = counts.astype(np.float64)
print(weights.dtype)
print(weights.itemsize, weights.nbytes)
```

astype在这里转换并建立新数组；itemsize为每元素字节数，nbytes为元素数据所占字节数。三个float64元素共 24 字节，不含 Python 对象元数据等额外成本。约百万个float64元素需要约 800 万字节，仅是数组数据本身的量级估算。

整数数组里写小数不会自动把整个数组改为浮点类型：

```
a = np.array([1, 2], dtype=np.int64)
a[0] = 1.9
print(a)
```

结果为[1 2]，小数部分被截去。用于平均、标准化与模型系数的数组应从开始就明确采用合适浮点类型，而不是等发现小数丢失后再转换。

Python 整数可按需要扩大表示，但 NumPy 固定宽度整数可能溢出。例如两个uint8数组[250]和[10]相加，结果按其范围回绕为[4]。图像像素常见 0–255 整数，直接做差或平方可能在计算前就出错。应先把输入转换成浮点，再相减或平方；在错误计算以后才转浮点，不能恢复已经丢失的信息。

配套数值接口先把实数输入转换为独立的float64数组，再执行差值与平方。以下辅助函数也会拒绝布尔、复数、字符串和一般对象数组；不能无说明地丢弃复数虚部或把文本解释为特征。

```
def as_float_array(values, name):
    array = np.asarray(values)
    if array.dtype.kind not in "iuf":
        raise ValueError(f"{name}: expected real numeric values")
    with np.errstate(over="raise", invalid="raise"):
        array = array.astype(np.float64)
    if not np.isfinite(array).all():
        raise ValueError(f"{name}: expected finite values")
    return array
```

`dtype.kind`中的`i`、`u`、`f`分别表示有符号整数、无符号整数和浮点类型。`astype`在这里生成副本，转换不修改调用者数组。`np.errstate`的局部异常检查将在数值稳定性一节说明。`float64`也不能精确保存任意大的整数；本实验输入是小尺度实数，不用于需要精确保留超大整数标识的任务。

规则数组与对象数组

`np.array([[1, 2], [3, 4]])`有规则矩形结构；`[[1, 2], [3]]`各行长度不等，不能直接作为常规二维数值数组。混入字符串可能使数组转为字符串类型，后续加减不再具有原数值意义。

`dtype=object`可以装入一般 Python 对象，但会回到逐对象操作，也带来第 3 章的内部可变对象共享问题。本章聚焦规则数值数组，不用对象数组绕开结构错误。数组的`copy()`复制数值缓冲区；若数组元素本身是对象引用，它不自动深复制这些对象。

zeros、ones、arange 与 linspace

```
print(np.zeros((2, 3)))
print(np.ones((1, 3)))
print(np.arange(1, 7, 2))
print(np.linspace(0.0, 1.0, 5))
```

前两者创建指定形状的全 0、全 1 数组；`arange`按步长生成并通常不含终点，得到 1、3、5；`linspace`指定点数，默认包含两端，得到 0、0.25、0.5、0.75、1。需要固定数量的等距浮点点时，`linspace`比依赖浮点步长恰好落在终点更清楚。

`np.full((n,), value)`创建指定值填充的数组，适合均值基线。`np.empty`分配数组但不把元素初始化为约定数值，其内容不能在完整赋值之前使用。`zeros_like(X)`沿用`X`的形状和通常的数据类型；若`X`是整数，不能假定新数组就适合存标准化小数。

4.4 索引、切片、布尔筛选与共享内存

二维索引与保留轴

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
print(X[1, 2])
print(X[0].shape)
print(X[:, 1].shape)
print(X[:, 1:2].shape)
print(X[0:1, :].shape)
```

依次为 6.0、(3,)、(2,)、(2, 1)和(1, 3)。单个整数索引选定该轴的一项并去除该轴；切片即使只选一项也保留该轴。二维写法X[i, j]直接说明两个轴的索引，通常比连续X[i][j]更容易核对。

> **例题 4.1：同一列的两种形状**
>
> X.shape == (5, 3)。预测X[:, 0]、X[:, 0:1]、X[0, :]、X[0:1, :]的形状，并说明哪些仍是二维数组。

解 依次为(5,)、(5, 1)、(3,)、(1, 3)。第二、四项仍有两个轴。若模型接口要求 $n \times d$ 输入，一个样本也应保留为(1, d)；若指标要求一维标签，就应有意取成(n,)，而非随意保留或去除轴。

基本切片的视图与显式复制

视图（view）是共享同一底层数据的数组表示，可以有不同形状、偏移和步长。数值数组的基本切片通常给出视图，与第 2 章列表切片建立新外层列表的行为不同。

```
a = np.array([[1., 2.], [3., 4.]])
view = a[:, :1]
independent = a[:, :1].copy()
view[0, 0] = 99
independent[1, 0] = -1
print(a)
print(independent)
print(np.shares_memory(a, view))
```

原数组变为[[99., 2.], [3., 4.]]；独立副本为[[1.], [-1.]]；共享检查为真。view is a虽为假，两者仍共享数据；身份不同不能推出数据互相独立。

a = np.asarray(source)用于取得数组表示；若输入已经是兼容类型的数组，它可能直接返回同一对象。需要保护调用者输入时，不应只写asarray就假定复制完成。np.array(source, copy=True)或转换后的copy()可以明确请求独立数值缓冲区。

视图可避免复制大块数据，但也会延长底层数据的存活。例如从很大的数组取得一个小切片，若仍引用原缓冲区，保留小切片可能继续占着大块内存。是否复制要根据共享语义与内存用途决定，不能一律认为“视图总更好”。

整数数组索引与布尔筛选

高级索引（advanced indexing）包括整数数组和布尔数组等形式。读取时通常产生新数组：

```
a = np.array([[1., 2.], [3., 4.], [5., 6.]])
picked = a[[0, 2]]
picked[0, 0] = 99
mask = a[:, 0] >= 3
print(mask)
print(a[mask])
print(a[0, 0])
```

布尔掩码为`[False True True]`，筛选后保留后两行；原`a[0, 0]`仍为 1.0。**掩码**（mask）在这里是决定保留哪些位置的布尔数组。多个条件使用`&`、`|`与`~`并分别加括号，例如`(a[:, 0] >= 3) & (a[:, 1] < 6)`。Python 的`and`不是逐元素数组逻辑操作。

直接赋值`a[mask] = 0`会修改原数组，因为它以原数组为赋值目标；不能根据“读取高级索引得到副本”推断这句也不修改原数组。`a[mask][:, 0] = 0`则先得到临时副本，再改副本，往往不改变`a`。避免依赖连续索引的隐含中间对象，直接指定清楚的赋值位置。

`X[[0, 1], [1, 2]]`按位置配对，取`X[0, 1]`和`X[1, 2]`，不是取两行两列的全部交叉组合。若要子矩阵，可以写`X[np.ix_([0, 1], [1, 2])]`。复杂索引先在小矩阵上核对实际值，再用于数据集。

4.5 逐元素运算、归约与轴的消去

逐元素运算与通用函数

逐元素运算（elementwise operation）按对应位置处理数值，例如`a + b`、`a * b`、`a ** 2`、`np.abs(a)`、`np.sqrt(a)`与`np.exp(a)`。形状相同时，每个输出位置只对应相同位置的输入；形状不同时需按下一节的广播规则处理。

NumPy 的许多数值函数属于**通用函数**（universal function，ufunc），以数组为单位实现逐元素操作。`math.sqrt`主要面向标量，不能把一个普通多元素数组直接当作标量；数组平方根用`np.sqrt`。函数名相似不意味着接口相同。

对真实值和预测均为`(n,)`的非空有限数组，可计算：

```
y = np.array([2., 4., 6.])
predicted = np.array([3., 4., 4.])
errors = predicted - y
mae_value = np.mean(np.abs(errors))
mse_value = np.mean(errors ** 2)
print(errors, mae_value, mse_value)
```

有符号误差为 1、0、−2，MAE 为 1，MSE 为 5/3，与第 1 章手算一致。若输入含非有限值，平均也可能成为 NaN 或无穷，因此指标入口仍需验证，不能只把循环替换

为一行表达式。

sum、mean 与 axis 的含义

归约（reduction）沿一个或多个轴汇总数值，从而消去相应轴。对 $X \in \mathbb{R}^{n\times d}$，沿轴 0 求和得到每个特征在所有样本上的总和：

$$s_j = \sum_{i=1}^{n} X_{ij}, \qquad \mathbf{s} \text{ 的数组形状为} (d,).$$

沿轴 1 求和则得到每个样本的特征总和：

$$t_i = \sum_{j=1}^{d} X_{ij}, \qquad \mathbf{t} \text{ 的数组形状为} (n,).$$

`axis`指出被汇总、消去的轴。不能把`axis=0`机械记成“每行”，它沿样本轴把各行合并，剩下列对应的结果。

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
print(X.sum(axis=0))
print(X.sum(axis=1))
print(X.mean(axis=0))
print(X.mean())
```

依次为`[5. 7. 9.]`、`[6. 15.]`、`[2.5 3.5 4.5]`和 3.5。未指定轴时，默认对全部元素汇总。不同物理单位的特征直接求总和通常没有任务意义，此处只是解释数组规则。

keepdims、布尔归约与位置

`keepdims=True`将被汇总轴保留为长度 1，而不是完全移除：

```
column_mean = X.mean(axis=0, keepdims=True)
row_mean = X.mean(axis=1, keepdims=True)
print(column_mean.shape)
print(row_mean.shape)
```

形状为`(1, 3)`和`(2, 1)`。该参数使结果更容易沿预期方向广播回原数组；它不是多计算几个均值，而是保留轴信息。

`np.isfinite(X)`返回逐位置布尔数组；`.all()`判断是否全部为真；`.any()`判断是否至少一项为真。`if X`对普通多元素数组会因真假含义不明确而报错，应根据意图写`X.size == 0`、`mask.any()`或`mask.all()`。

`argmax`与`argmin`返回位置，不返回最大或最小值。`X.argmax(axis=1)`给出每行最大值的列下标；默认不指定轴时按展平顺序给出位置，同分时通常返回首先遇到者。若用该位置关联其他字段，应明确它属于哪个轴，不能把局部列下标当成样本编号。

例题 4.2：按列中心化的手算

对 $X=\left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，先求每列均值，再从每个元素减去所属列均值。写出结果、代码和结果均值。

解　列均值为 $(2.5, 3.5, 4.5)$。两行分别减去同一均值行，得到

$$X_c=\begin{pmatrix}-1.5&-1.5&-1.5\\1.5&1.5&1.5\end{pmatrix}.$$

代码为`Xc = X - X.mean(axis=0, keepdims=True)`。`Xc.mean(axis=0)`应在浮点容差内为零。若改为`axis=1`，就减去每行自身均值，得到两行均为 $(-1, 0, 1)$，这是另一种操作，不能因同样叫“减均值”就混用。

4.6　广播：不同形状怎样逐元素配对

从标量到逐列操作

`X + 1`把同一个标量加到每个元素上，不需要先手工创建一个全 1 数组。对形状`(n, d)`的`X`，加上形状`(d,)`的`offset`，常用于给每个特征加不同常数。这个规则称为广播（broadcasting）。

广播规定下标怎样对应，不必真的把较小输入复制成大数组。但是运算的输出和某些中间量仍可能很大。输入不复制与整个表达式不消耗内存是不同判断。

规则：从右向左比较各轴

对逐元素运算，将各数组的形状从右侧对齐。每一对轴长度必须相同，或其中一个为 1；缺少的前导轴按长度 1 理解。兼容时，长度 1 的轴可在逻辑上重复使用，输出保留对应的非 1 长度；相同长度则保持该长度。存在一对不相等且都不为 1 的轴，就不能按该广播规则运算。

本章实例轴长均为正。长度为 0 的数组另需按接口处理，不能先用广播绕过非空验证。

对`(2, 3)`与`(3,)`，先把后者理解为`(1, 3)`：最后轴 3 与 3 相同，前轴 2 与 1 兼容，输出`(2, 3)`。对`(2, 3)`与`(2,)`，右侧 3 与 2 冲突，因此失败。若要每个样本加一个值，应把后者明确变为`(2, 1)`。

输入形状	是否兼容	输出形状
(5, 3)与(3,)	是	(5, 3)
(5, 3)与(5, 1)	是	(5, 3)
(5, 1)与(1, 3)	是	(5, 3)
(5, 3)与(5,)	否	末轴 3 与 5 冲突
(2, 1, 3)与(4, 3)	是	(2, 4, 3)

广播只认识轴的位置和长度，不认识“这个轴是样本还是特征”。若 $n = d$，给(n, d)加(n,)可以运行，但会沿最后特征轴配对，可能偏离逐样本加偏移的意图。测试时刻意使用不同的样本数和特征数，有助于让此类错误暴露。

用下标解释扩展方向

设 $X \in \mathbb{R}^{n\times d}$，每个特征有偏移 a_j，则

$$Z_{ij} = X_{ij} + a_j.$$

这里 a 不依赖样本下标 i，因此用a.shape == (d,)或(1, d)。若每个样本有偏移 b_i，则

$$U_{ij} = X_{ij} + b_i,$$

此时 b 不依赖特征下标 j，应显式写成(n, 1)。

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
feature_offset = np.array([10., 20., 30.])
sample_offset = np.array([100., 200.])
print(X + feature_offset)
print(X + sample_offset[:, None])
```

第一个结果为两行 $(11, 22, 33)$ 和 $(14, 25, 36)$；第二个结果为 $(101, 102, 103)$ 和 $(204, 205, 206)$。看下标中哪个符号缺失，比只背“横着广播”“竖着广播”更能推广到多轴数据。

静默错误：标签向量与预测列数组

例题 4.3：零误差为何变成 4/3

真实值与预测定义如下。两者数值看似相同，计算均方误差为什么不是 0?

```
y = np.array([1., 2., 3.])
p = np.array([[1.], [2.], [3.]])
print(np.mean((p - y) ** 2))
```

解 p.shape == (3, 1),y.shape == (3,)按广播变为(1, 3)。相减的输出为(3, 3)

，元素是 $p_i - y_j$，而非只计算 $p_i - y_i$：

$$p - y = \begin{pmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix}.$$

九个平方的和为 12，平均得到 $12/9 = 4/3$。想要的有符号误差只对应主对角线上的三个 0；其余六项比较了不同样本。

若接口规定一维标签和预测，应先检查`p.shape == y.shape`，本例有意转换为`p[:, 0]`后再计算。若接口采用二维列数组，则应将`y`转换成`y[:, None]`。无条件`reshape(-1)`可能把多输出预测也压成一条长向量，掩盖原结构，因此形状调整必须有语义依据。

更一般地，若 $p_i = y_i$，却把所有两两差都计算出来，则

$$\begin{aligned} \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}(y_i - y_j)^2 &= \frac{2}{n}\sum_{i=1}^{n} y_i^2 - 2\left(\frac{1}{n}\sum_{i=1}^{n} y_i\right)^2 \\ &= 2\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2\right]. \end{aligned}$$

第一步展开平方，并分别对 i,j 求和；第二步使用 $\bar{y} = n^{-1}\sum_i y_i$。错误结果等于以分母 n 定义的数据方差的两倍，本例方差为 $2/3$。这个推导也说明：只有全体标签相同时，该错误才可能偶然返回 0。

4.7 矩阵乘法与维度核对

从一个样本的加权和到批量预测

第 1 章的直线模型是 $\hat{y} = wx + b$。多个特征时，用每个特征各自的权重做加权和：

$$\hat{y}_i = \sum_{j=1}^{d} X_{ij} w_j + b.$$

X_{ij} 是第 i 个样本的第 j 个特征；w_j 是该特征权重；b 是所有样本共用的偏置。若目标单位为元，w_j 的单位应为元除以对应特征单位，各乘积才具有相同单位。

矩阵乘法（matrix multiplication）把上述“沿特征求乘积和”写成

$$\hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}_n.$$

数学中 $\mathbf{w} \in \mathbb{R}^d$，$\mathbf{1}_n$ 表示 n 个 1；NumPy 采用`X.shape == (n, d)`、`w.shape == (d,)`时，`X @ w`得到`(n,)`，再把标量`b`广播到每个样本。符号`@`表示矩阵乘法，`*`仍是逐元素乘法。

例题 4.4：乘积和与逐元素乘积

设

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad b = 4.$$

计算批量预测；比较`X @ w`、`X * w`与`(X * w).sum(axis=1)`。

解 三行加权和分别为 $2-2=0$、$6-4=2$、$10-6=4$，加偏置得到 $(4,6,8)$。`X @ w` 为长度 3 数组 $(0,2,4)$；`X * w`广播后为

$$\begin{pmatrix} 2 & -2 \\ 6 & -4 \\ 10 & -6 \end{pmatrix},$$

形状为`(3, 2)`。随后沿特征轴 1 求和才得到 $(0,2,4)$。两个表达式在实数运算中相同，浮点求和顺序不同可能带来微小差异。

```
X = np.array([[1., 2.], [3., 4.], [5., 6.]])
w = np.array([2., -1.])
print(X @ w + 4.0)
print((X * w).sum(axis=1) + 4.0)
```

两个矩阵的乘法与单轴特例

若 $A \in \mathbb{R}^{m\times d}$、$B \in \mathbb{R}^{d\times k}$，乘积 $C = AB \in \mathbb{R}^{m\times k}$ 定义为

$$C_{i\ell} = \sum_{j=1}^{d} A_{ij} B_{j\ell}.$$

内侧长度 d 必须相等，结果保留外侧长度 m, k。代码`A @ B`按此规则运算。一般 $AB \neq BA$；即使两个方向都能计算，也可能代表不同含义。

左、右操作数形状	@结果形状	含义
`(n, d)`与`(d,)`	`(n,)`	每行与同一权重做内积
`(n, d)`与`(d, 1)`	`(n, 1)`	保留二维列结果
`(n, d)`与`(d, k)`	`(n, k)`	每个样本产生 k 个输出
`(d,)`与`(d,)`	`()`	两个一维数组内积

一维数组的@具有专门的升轴再去轴规则，不必把`(d,)`强行解释成既是行又是列。多轴`matmul`将最后两个轴作为矩阵轴、前导轴作为批次轴并按规则广播；本章实验主要采用一维与二维，后续批量神经网络再展开多轴乘法。

`np.dot`在一维、二维常见情形与熟悉的内积、矩阵乘法有关，但高维规则与`matmul`并不完全相同。本书矩阵乘法优先写`@`，减少在尚未核对高维语义时混用名称。

带形状的计算记录

对每个表达式同时写出输入和输出，比报错以后随机转置更有效。例如：

$$
\begin{aligned}
X : (n, d), \quad W : (d, k), \quad b : (k,), \\
S = XW : (n, k), \\
Z = S + b : (n, k), \\
\operatorname{mean}(Z, \text{axis} = 0) : (k,).
\end{aligned}
$$

W 的每一列对应一个输出的权重；b 的每项对应一个输出的偏置。若 b 表示每个样本的偏置，则应采用`(n, 1)`，语义已经改变。线性代数公式与数组形状相互核对，第 6 章将用同一习惯检查投影、最小二乘与矩阵微分。

4.8 随机数、固定种子与复现条件

生成器、状态与种子

计算机中的常用随机数是**伪随机数**（pseudorandom numbers）：由算法和内部状态生成，统计表现类似所需分布，但给定相同条件可以重建序列。**种子**（seed）用于初始化状态，不是每次抽样都重新计算一次的固定结果。

```
rng = np.random.default_rng(2026)
a = rng.normal(size=(2, 3))
b = rng.normal(size=(2, 3))
print(a.shape, b.shape)
```

`default_rng`创建随机数生成器对象；`normal`默认从均值 0、标准差 1 的正态分布抽样。同一个生成器连续调用会推进状态，`a`与`b`通常不同。每次都重新创建同种子生成器，则会反复得到序列起点，可能把本应独立的扰动变成重复数据。

常用方法包括`rng.integers(low, high, size=...)`生成下界包含、上界通常不包含的整数；`rng.uniform`生成均匀抽样；`rng.permutation(n)`返回 0 至 $n-1$ 的随机排列。分布含义在第 8 章系统说明，本章只用它们产生可核对的教学输入。

配对打乱必须使用同一个排列

```
X = np.array([[1., 10.], [2., 20.], [3., 30.]])
y = np.array([100., 200., 300.])
rng = np.random.default_rng(2026)
order = rng.permutation(len(X))
X_shuffled, y_shuffled = X[order], y[order]
print(X_shuffled[:, 0] * 100 == y_shuffled)
```

三个布尔结果都应为真，因为两边使用同一个排列。若分别打乱特征和标签，两

次随机调用会推进状态，可能采用不同排列并破坏对应关系。同一种子并不意味着同一个生成器的两次调用得到相同结果。

打乱不自动意味着合理划分。时间任务、用户分组任务仍需第 1 章的约束；随机抽样会影响哪些样本进入各集合，应保存最终索引或标识。只有保存种子而未保存原始行顺序，输入顺序一变就可能得到另一组划分。

相同种子的适用范围

固定生成器类型、实现版本、种子和调用顺序，可重复生成相同序列；NumPy 生成器不承诺所有未来版本和环境下完整随机流永远不变。跨环境复现应记录 NumPy 版本、生成器类型以及最终数据或索引。不同算法调用可能消耗不同数量随机数，因此在前面增加一个随机抽样会影响后续结果。

可把划分与噪声生成分别交给独立生成器，减少无关修改的相互影响。对大型项目还可能有多个进程、不同库和硬件中的随机状态，第 25 章与第 44 章再展开。本章实验 4.2 采用固定训练、验证、测试数据，随机种子只用于数值性能对照输入，不把它当成划分记录。

4.9 数值稳定性与浮点核对

范围、舍入与非有限结果

数学实数不受计算机位数限制，浮点数则只能表示有限范围内离散的近似数。有限输入经过运算仍可能溢出，例如大数平方或大指数；极小正数也可能下溢到 0。输出警告不一定使程序立即停止，后续计算可能继续传播无穷或 NaN。

`np.isfinite(result).all()`检查结果是否全部有限。若问题本来不允许无穷，应在适当边界检查并停止，而非用`nan_to_num`无说明地替换成 0。缺失值处理是数据规则，数值溢出是计算问题，不能靠同一种替换掩盖。

`np.errstate`可以在局部改变浮点异常的处理方式：

```
with np.errstate(over="raise", invalid="raise", divide="raise"):
    result = np.exp(np.array([1000.]))
```

这段故意演示失败路径，在常见`float64`环境中会抛`FloatingPointError`。它帮助尽早发现失效位置，但不自动修复计算方法。不要把故意失败片段直接加入必须从头成功完成的实验入口。

先减最大值再取指数

某些模型要把一组实数分数转成非负且和为 1 的数。设 $z_1, \dots, z_k$ 为有限分数，直接定义

$$p_j = \frac{\exp(z_j)}{\sum_{\ell=1}^{k} \exp(z_\ell)}.$$

这里先把它作为数值归一化规则；第 13 章和第 30 章再讲其模型意义。分数为 1000、1001、1002 时，直接取指数可能溢出。令 $M = \max_j z_j$，分子分母同时除以 $\exp(M)$，得

到实数意义下等价的表达式

$$p_j = \frac{\exp(z_j - M)}{\sum_{\ell=1}^{k} \exp(z_\ell - M)}.$$

所有指数输入都不大于 0，最大项指数为 1，分母至少为 1，可避免正向指数溢出。非常小的项仍可能下溢到 0，但不会令所有项同时为 0。

例题 4.5：稳定归一化与按行计算

一批数据有两行分数 $(1000, 1001, 1002)$ 和 $(-1000, -1000, -1000)$。写出稳定计算，说明每个中间数组形状，并给出近似结果。

```
z = np.array([[1000., 1001., 1002.], [-1000., -1000., -1000.]])
shifted = z - z.max(axis=1, keepdims=True)
exp_values = np.exp(shifted)
p = exp_values / exp_values.sum(axis=1, keepdims=True)
print(p)
print(p.sum(axis=1))
```

解 行最大值形状`(2, 1)`，广播减法后`shifted`为`(2, 3)`，两行分别为 $(-2, -1, 0)$ 和 $(0, 0, 0)$。指数数组也为`(2, 3)`，行分母为`(2, 1)`。结果近似为

$$\begin{pmatrix} 0.090031 & 0.244728 & 0.665241 \\ 1/3 & 1/3 & 1/3 \end{pmatrix},$$

各行和为 1。归一化得到和为 1 的数，不自动证明它们是经过校准的真实发生概率。若输入整行含无穷或 NaN，以上有限性前提不成立。

接近数相减与运算顺序

两个很大且非常接近的数相减，会放大输入近似误差对差值的影响。后文常用恒等式

$$\|\mathbf{x} - \mathbf{q}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{q}\|^2 - 2\mathbf{x}^{\mathrm{T}}\mathbf{q}$$

计算距离，但右侧可能在大数之间发生抵消，得到微小负数，甚至丢失应有的小距离。将负值截到 0 能修正部分舍入表现，却不能恢复已经损失的精度。对规模小且近邻差异细的教学数据，本章完整实验优先直接计算差后平方。这种形式减少抵消风险，却仍会在差值或平方过大时溢出，必须另外检查。

求和顺序也影响浮点结果。数学加法满足结合律，浮点近似加法不保证严格满足。循环、向量化和底层矩阵库可能采用不同顺序，所以应比较合理误差，而不要求所有位完全相同。若差异远大于容差，应先检查类型、形状和公式，再讨论舍入。

绝对容差、相对容差与先查形状

常用近似检查可写为

$$|a - b| \le \mathrm{atol} + \mathrm{rtol}|b|.$$

atol是绝对容差，适合接近 0 的量；rtol是相对容差，随参考量 b 的尺度变化。np.allclose按类似规则比较数组，它还可能广播，因此检查前应先确认形状相同。

```
if loop_result.shape != vector_result.shape:
    raise ValueError("result shapes differ")
np.testing.assert_allclose(
    loop_result, vector_result, rtol=1e-12, atol=1e-12
)
```

np.testing.assert_allclose失败时会给出差异信息，便于实验核对。上述容差适用于本章小尺度float64计算，并非所有模型通用精度标准。float32、长求和或病态问题可能需要不同容差，容差也不能大到掩盖公式错误。含 NaN 的数据首先应按任务规则拒绝，不把“两个地方都 NaN”当成正确一致。np.testing.assert_allclose默认允许相同位置的 NaN 通过核对；需要明确拒绝时，可先检查np.isfinite或设置equal_nan=False。

4.10 从逐样本循环到向量化

保留一个可手算的循环实现

对批量线性预测，先写直接对应公式的循环：

```
def predict_loop(X, w, b):
    result = []
    for row in X:
        value = b
        for feature, weight in zip(row, w):
            value += feature * weight
        result.append(value)
    return np.array(result)
```

该教学核心假定X为非空有限二维数组、每行长度等于len(w)、w为一维、b为有限标量；通用接口应先验证。zip仍会截短不等长输入，因此数组并未让第 2 章的配对问题消失。

计算顺序为对每个样本建立一个累加器，依次乘每个特征权重，最后保存预测。时间量级为 $O(nd)$，结果空间为 $O(n)$。循环实现保留为可独立对照的参考，不必为了代码短而立即删除。

先去掉内层循环，再去掉外层循环

第一步让 NumPy 完成一行的逐元素乘积与归约：

```
result = []
for row in X:
    result.append(np.sum(row * w) + b)
result = np.array(result)
```

第二步让全部样本一起计算：

```
result = np.sum(X * w, axis=1) + b
```

形状为(n, d)的X乘(d,)的w，先广播得到每样本每特征贡献，再沿轴 1 归约。第三步利用矩阵乘法表达同一加权和：

```
result = X @ w + b
```

两种批量表达式都保留 $O(nd)$ 算术量级；速度改进主要来自执行方式和内存访问，而不是从 nd 次计算变成只算一次。X * w通常建立 $n \times d$ 的中间结果；矩阵乘法可避免显式存储这份乘积，并可能调用优化数值实现。

评价向量化的正确顺序

先用例题 4.4 的三行数据核对三个实现值与形状，再生成较多数据比较误差，最后计时。**基准测试**（benchmark）是在规定输入与环境下测量性能；首次调用可能受到初始化、缓存等影响，因此应先预运行，再重复测量，报告中位数或其他明确统计量。

time.perf_counter()提供适合测量间隔的计时值。计时时不要让一个实现同时读取文件、另一个只做计算；不要让一个实现包含打印、另一个没有。多线程数值库、后台负载、数据形状和规模都会影响实际耗时，本章不把固定加速倍数作为通过条件。

np.vectorize提供便于把标量函数应用到数组的接口，但主要仍采用 Python 层循环，不能仅凭名称认为它等同于高效数值向量化。对小输入、复杂分支或逐文件处理，普通循环往往更清楚；对共享运算规则的大批数值，数组操作更自然。

4.11 成对距离：广播的用途与内存代价

从一个查询到全部查询

设训练数组 $X \in \mathbb{R}^{n \times d}$，查询数组 $Q \in \mathbb{R}^{m \times d}$。第 a 个查询到第 i 个训练样本的平方欧氏距离为

$$D_{ai} = \sum_{j=1}^{d} (Q_{aj} - X_{ij})^2, \qquad D \in \mathbb{R}^{m \times n}.$$

欧氏距离（Euclidean distance）是各坐标差的平方和再开平方；比较最近顺序时，平方根是单调的，可以直接比较平方距离，节省一次开方。各特征量纲不同会影响距离，下一节实验先用训练统计量缩放。

构造Q[:, None, :]，形状为(m, 1, d)；构造X[None, :, :]，形状为(1, n, d)。广播后得到全部三重下标的差：

```
delta = Q[:, None, :] - X[None, :, :]
distance2 = np.sum(delta ** 2, axis=2)
```

delta[a, i, j]对应 $Q_{aj} - X_{ij}$，沿轴 2 归约去掉特征，留下查询和训练样本两个轴。若误写axis=1，就把训练样本轴求和，输出(m, d)，不能用于逐查询找训练邻居。

例题 4.6：三维中间数组的手算

训练样本为 $(0,0),(2,0),(0,2)$，查询为 $(1,0),(0,3)$。写出平方距离矩阵和最近样本下标。同分时取训练顺序靠前者。

解 第一个查询到三点的平方距离为 1、1、5，第二个为 9、13、1。因此

$$D = \begin{pmatrix} 1 & 1 & 5 \\ 9 & 13 & 1 \end{pmatrix}.$$

两个查询对应最近下标 0、2，使用 Python 从 0 开始的编号。中间`delta`形状为`(2, 3, 2)`，归约后为`(2, 3)`。`distance2.argmin(axis=1)`得到每行最小距离的训练下标，不应沿轴 0 找“每个训练样本最近的查询”。

k 个近邻、排序与索引标签

若每个查询取距离最小的 k 条训练标签平均，可写：

```
order = np.argsort(distance2, axis=1, kind="stable")[:, :k]
predicted = y_train[order].mean(axis=1)
```

`argsort`返回排序后的下标，`kind="stable"`使距离同分时保持训练原顺序；`order.shape == (m, k)`。`y_train`形状为`(n,)`，用二维整数下标取得`(m, k)`标签数组，沿近邻轴 1 平均，得到`(m,)`预测。

本章采用完整排序，距离计算需 $O(mnd)$，排序通常需 $O(mn\log n)$ 时间，距离矩阵需 $O(mn)$ 空间，显式差数组需 $O(mnd)$ 空间。完整近邻算法、部分选择与检索结构在第 15 章讨论；本章重点在于公式与轴对应。

批次计算与更省内存的恒等式

若 $m = n = 10000$、$d = 100$，`delta`含 10^{10} 个元素，单独一份`float64`数据约需 80 GB，平方等中间结果还会增加峰值。输入数组本身不算大，并不表示成对广播输出也能放入内存。

批次（batch）把查询分成若干小组处理，组内向量化，组间循环。若每组最多 B 个查询，差数组由 mnd 缩为 Bnd，全部距离运算的总量级不变。对于只需要预测的任务，每组完成后可只保留其预测，不存全部距离矩阵。

```
blocks = []
for start in range(0, len(Q), block_size):
    part = Q[start:start + block_size]
    blocks.append(predict_vectorized(X, y_train, part, k))
predicted = np.concatenate(blocks)
```

此片段假定查询非空、批大小为正整数，完整答案补充边界检查。`concatenate`沿已有轴连接预测，不额外增加批次轴；`stack`则增加新轴，适用于等形数组的另一种组织目的。

也可按恒等式写出距离矩阵：

```
distance2 = (
    np.sum(Q ** 2, axis=1, keepdims=True)
    + np.sum(X ** 2, axis=1)[None, :]
    - 2 * (Q @ X.T)
)
```

三项分别为`(m, 1)`、`(1, n)`和`(m, n)`，可避免显式`(m, n, d)`差数组。但仍需存储 $m \times n$ 结果，并有前述大数抵消风险。优化应同时考虑计算量、峰值内存与数值条件，不能只比较代码行数。

4.12 实验 4.1：形状、共享内存与稳定计算

入口为配套实验`/ch04_arrays.py`，依赖 NumPy，无需下载数据。实验使用 2 行 3 列的数组，依次检查形状属性、按轴归约、按列中心化、视图修改、高级索引副本、错误广播和稳定归一化。每一步采用前面已手算的数值，因此可先预测再执行。

```
python3 ch04_arrays.py
```

关键输出为：

```
shape ndim size: (2, 3) 2 6
column shapes: (2,) (2, 1)
sum axis=0: [5. 7. 9.]
sum axis=1: [ 6. 15.]
shares: True False
wrong residual shape: (3, 3)
wrong MSE: 1.3333333333333333
correct MSE: 0.0
same seed same draws: True
all checks passed
```

中心化结果为两行全 -1.5 和全 1.5；通过视图修改后原数组第一项成为 99，高级索引副本上的修改不再影响原数组。稳定归一化输出应与例题 4.5 一致，各行和与 1 之差不超过 10^{-12}。

脚本使用`np.set_printoptions`统一小数显示，这只改变数组的显示方式，不改变计算精度。`suppress=True`减少小数显示为科学计数法，并不会把小数截为 0。屏幕显示 0 可能只是精度设置下的舍入，应通过差值或更高显示精度判断。

修改任务如下：把列切片`a[:, :2]`改成`a[:, :2].copy()`，核对原数组不再受影响；把行分母中的`keepdims=True`去掉，分析为什么`(2, 3)`除`(2,)`失败；把归一化改成`axis=0`，说明此时哪一方向和为 1；把预测列数组改为一维，核对错误 MSE 归零。每次修改只处理一个因素并恢复原版，避免前面的原地修改影响后续手算。

4.13 实验 4.2：训练缩放、批量近邻与性能对照

任务、数据与假设

入口为配套实验`/ch04_numeric.py`，只依赖 Python 标准库和 NumPy，CPU 可运行。任务是以 3 个数值特征预测连续目标，比较 $k = 1$ 与 $k = 3$ 的近邻平均。第一特征可理解为一个主要数值量，第二特征用于演示不同尺度，第三特征恒为 1，用于检查零标准差处理；三者与目标均是教学构造，不声称真实因果关系。

训练数据为

$$X_{\text{train}} = \begin{pmatrix} 0 & 10 & 1 \\ 1 & 20 & 1 \\ 2 & 10 & 1 \\ 3 & 20 & 1 \end{pmatrix}, \qquad \mathbf{y}_{\text{train}} = (4, 6, 8, 10).$$

验证与测试数据如下：

集合	特征 1	特征 2	特征 3	目标
验证	0.2	11	1	4.4
验证	1.2	19	1	6.4
验证	2.2	11	1	8.4
验证	2.8	19	1	9.6
测试	0.4	12	1	4.8
测试	1.6	12	1	7.2
测试	2.4	12	1	8.8
测试	3.5	22	1	11.0

数组形状都为`(4, 3)`，标签为`(4,)`。这些集合预先固定，不在本实验中重新随机划分。候选按验证 MAE 选择，同分时取声明顺序靠前的 k；同距离近邻按训练顺序；不合并验证集重训。流程与实验 3.2 一致，但数组计算和预处理多了一层形状要求。

只从训练数组估计标准化参数

标准化（standardization）在本节指对每个特征减训练均值，再除以训练标准差。它把尺度不同的特征转换成更可比较的无量纲数值，但不自动去除相关性，也不保证任何算法都会变好。对第 j 列定义

$$\mu_j = \frac{1}{n} \sum_{i=1}^{n} X_{ij},$$

$$s_j = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (X_{ij} - \mu_j)^2},$$

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\tilde{s}_j}, \qquad \tilde{s}_j = \begin{cases} s_j, & s_j > 0, \\ 1, & s_j = 0. \end{cases}$$

这里采用分母 n，作为本实验确定的缩放规则，对应`std(ddof=0)`。`ddof`为分母自由度修正量，设为 1 时分母变为 $n-1$，统计估计含义在第 8 章讨论。不要在训练和预测中采用不同约定。

训练列均值为 $(1.5, 15, 1)$；第一列平方离差为 2.25、0.25、0.25、2.25，均值 1.25，因此标准差 $\sqrt{1.25} \approx 1.118034$；第二列标准差 5；第三列标准差 0，使用替代尺度 1。训练后第三列标准化值全为 0。

```
mean = X_train.mean(axis=0, keepdims=True)
std = X_train.std(axis=0, ddof=0, keepdims=True)
scale = np.where(std == 0, 1.0, std)
Z_train = (X_train - mean) / scale
Z_val = (X_val - mean) / scale
Z_test = (X_test - mean) / scale
```

`mean`、`std`和`scale`形状均为`(1, 3)`，广播到各集合。`np.where(condition, a, b)`逐位置选择两个输入中的值；此处在除法之前建立安全尺度。若先计算含零除的表达式，再放入`where`，可能已经触发无效除法，因为普通函数参数通常会先求值。

均值与尺度只从训练数据估计，验证和测试复用它们。若分别给各集合减自身均值、除自身标准差，单个新样本就无法按训练规则定位，评价条件也发生变化。若在全部数据上估计，再切出集合，则把后续输入信息用于训练预处理，形成第 1 章所述泄漏。

手算一个查询的距离

第一条验证查询为 $(0.2, 11, 1)$。同一训练缩放下，查询到第 i 条训练点的平方距离可以直接写为

$$D_i = \frac{(0.2 - X_{i1})^2}{1.25} + \frac{(11 - X_{i2})^2}{25} + (1 - X_{i3})^2.$$

同一均值在相减中抵消，但尺度仍影响各坐标的权重。四个距离依次为 0.072、3.752、2.632、9.512，排序下标为 0、2、1、3。

$k = 1$ 预测 4，对真实 4.4 的绝对误差 0.4；$k = 3$ 平均前三条标签 $(4 + 8 + 6)/3 = 6$，误差 1.6。这里第二特征的尺度被缩到与第一特征较接近；若直接用原数值距离，第二特征差异会获得不同权重。

训练常数列在当前各集合中均为 1，因此距离贡献为 0。若某个查询在该列变为 $q_j \neq 1$，它到每条训练记录都会增加相同的 $(q_j - 1)^2$：距离数值改变，但对这个查询的近邻排序不变。该变化仍应检查来源，不能从训练列恒定推断未来也必然恒定。第 10 章再讨论常数特征删除及完整预处理流程。

循环与向量化实现的对应

完整入口保留predict_loop和predict_vectorized。前者三层循环计算查询、训练样本和特征；后者用前节广播形成距离矩阵，再稳定排序取得 k 个标签平均。两者采用同一缩放结果和同分规则，避免把算法条件差异误认为数值实现差异。

向量化接口在计算前检查训练与查询均为非空有限二维数组，特征数相同，训练标签恰为(n,)且有限，k 为 1 至 n 的整数。布尔值虽然在 Python 中与整数有关系，本接口不将True当作合法 k。两个实现共用输入验证，先调用前面的as_float_array，避免整数差值回绕。缩放、距离、标签平均和指标计算均在局部np.errstate检查下执行；有限输入若使中间量溢出，会抛FloatingPointError，不会继续把全无穷距离按同分规则选为某个正常预测。遇到这类输入应检查尺度与算法，不能用全局关闭警告代替修复。这些检查不能因底层矩阵运算可能报错就省略，因为有些错误会广播成功。

评价函数同样检查预测与标签均为等形一维非空有限数组：

```
def checked_mae(y, p):
    y = as_float_array(y, "y")
    p = as_float_array(p, "p")
    if y.ndim != 1 or y.size == 0 or p.shape != y.shape:
        raise ValueError("expected paired nonempty 1D arrays")
    with np.errstate(over="raise", invalid="raise", divide="raise"):
        return float(np.mean(np.abs(p - y)))
```

输入先转浮点，才能避免无符号整数减法回绕；例如uint8真实值 255、预测 0 的绝对误差应为 255，直接在原类型中相减却可能得到 1。末尾的float只把 NumPy 数值转换为普通 Python 浮点，便于 JSON 保存，不会提高精度。

运行与预期结果

在配套实验目录运行：

```
python3 ch04_numeric.py
python3 ch04_solutions.py
```

主要输出应为：

```
mean: [[1.5, 15.0, 1.0]]
scale: [[1.118033988749895, 5.0, 1.0]]
validation k=1: 0.400000
validation k=3: 1.466667
selected k: 1
test predictions: [4.0, 8.0, 8.0, 10.0]
test MAE: 0.850000
test baseline MAE: 2.050000
```

$k = 3$ 的完整逐条结果如下，必须按每条查询的距离排序分别取得近邻，不能从第一条查询类推其他样本。

验证行	真实值	$k = 1$ 预测	$k = 3$ 预测	$k = 3$ 绝对误差
1	4.4	4	6	1.6
2	6.4	6	8	1.6
3	8.4	8	22/3	16/15
4	9.6	10	8	1.6

表中 $k = 3$ 误差总和为 88/15，平均 $22/15 \approx 1.466667$，因此按最小验证 MAE 选择 $k = 1$。

所选 $k = 1$ 测试误差为 0.8、0.8、0.8、1.0，平均 0.85。预先声明的训练均值为 7，测试 MAE 为 2.05。不要因这两个最终分数恰与实验 3.2 相同，就认为两份数据和内部距离相同；本实验的训练输入有 3 个特征，最近记录的选择依据也不同。

性能对照与保存结果

同一入口另用种子 2026 生成`(12000, 12)`的正态数组和长度 12 的权重，比较`linear_loop`与`X @ w + 4.0`。先核对输出形状与数值，采用`rtol=1e-12`、`atol=1e-12`，再预运行并各计时 3 次，报告耗时中位数。最大绝对差通常处于 10^{-14} 量级或更小，实际值随数值库和环境变化。

计时不是验收正确性的条件；在小规模、不同线程或机器负载下，速度比会变化。实验不下载数据、不训练大型网络，也不要求 GPU。数值对照回答的是相同公式的两种实现是否一致，不能回答哪种模型泛化更好。

结果目录`ch04_results`包含`summary.json`与`arrays.npz`。JSON 保存版本、种子、均值、尺度、验证指标、选择、测试指标和计时；NPZ 是 NumPy 保存多个命名数组的格式，保留数据类型和形状，适合本实验数值输入与预测。

```
with np.load("ch04_results/arrays.npz", allow_pickle=False) as saved:
    train = saved["train"]
    predictions = saved["predictions"]
    print(train.shape, predictions.shape)
```

这个相对路径示例假定工作目录为配套实验目录。`allow_pickle=False`表示按普通数值数组读取，不启用 Python 对象序列化支持。实际入口仍以自身路径定位输出，避免启动目录影响结果。

JSON 中的计时自然可能每次不同，随机生成数组也应结合版本和调用顺序理解。训练、验证和测试是固定数组，近邻结果应在上述容差内一致。输出文件使用同名覆盖，若上一轮失败而旧文件仍存在，应检查本轮退出状态与日志，不靠“结果文件存在”判断运行成功。

修改任务与结果解释

1. 把标签改为`y_train[:, None]`传给向量化函数，接口应主动拒绝；不要让`mean(axis=1)`在增加的多余轴上偶然返回数值。
2. 删除标准化，比较第一条查询的邻居顺序，解释各坐标差对距离的权重；这是新的预处理方案，应重新完成验证选择，不能直接挑测试指标好的版本。
3. 把第三列全改为 5 并保持各集合一致，均值与尺度记录应相应改变，但该列标准化仍为 0，预测应保持一致。
4. 将最近邻预测按查询分块，批大小分别为 1、2 和大于查询数的 10，核对与完整批量一致。配套答案给出完整实现。
5. 只为数值核对，将 X 和权重改为`float32`，观察差异并使用与精度相称的容差；保留原版`float64`结果供比较，不把不同类型混作同一计时条件。

4.14 常见错误、知识联系与自检

NumPy 程序常见问题可以按信息缺失的位置定位。类型错误要先看`dtype`和转换发生在运算前还是后；形状错误要逐轴核对样本、特征和输出含义；结果被意外修改要检查视图、别名与原地赋值；随机结果变化要检查生成器状态、调用顺序和数据行顺序；数值异常要检查尺度、有限性和运算形式。

“没有报错”只排除了某些不兼容情况。`(n, 1)`与`(n,)`相减会成功，输入输出同为方阵时错误转置也可能成功。因此核对包括：能手算的小输入、不同轴长、预测与标签的严格等形检查、原输入是否改变，以及与独立循环实现的比较。

本章把第 2 章的累加器写成归约，把嵌套循环写成广播与矩阵乘法，把第 3 章引用问题扩展为共享内存。第 5 章会用 pandas 整理带字段名的表格，第 6 章会进一步解释矩阵几何；表格字段名、数组位置和数学符号需要明确对应，不能转换一次就丢掉特征顺序。

自检应能逐步说明：为什么`X.mean(axis=0)`得到每列均值；为什么一维`w.T`没有变成列数组；为什么数组切片后修改可能影响原数据；为什么固定种子不足以保证任意修改后的随机流一致；为什么把距离计算写成一行仍可能用尽内存；为什么标准化参数必须只由训练集确定。

4.15 分层习题

以下编号为 4.1–4.16。数组题同时写值与形状，复杂度题区分输入、输出与中间量。

4.1 **概念。**解释`shape`、`ndim`、`size`和`dtype`。200 个样本、每个样本 10 个特征，数组维数与特征空间维数分别是多少?

4.2 **形状。**令`a = np.arange(12).reshape(3, 4)`，写出`a[1]`、`a[:, 1]`、`a[:, 1:2]`、`a[1:2]`、`a.T`的形状；比较`a.T`和`a.reshape(4, 3)`的第一行。

4.3 **类型。**整数数组`[1, 2]`第一项赋 1.9 后是什么？为什么对像素整数先平方再转浮点不能保证正确?

4.4 **共享**。令`a = np.arange(6).reshape(2, 3)`，`b = a[:, :2]`，`c = a[[0, 1]]`。依次修改`b[0, 0] = 99`、`c[1, 1] = -1`，写出`a`，并说明身份比较能否替代共享内存检查。

4.5 **归约**。对 $A = \left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，求沿轴 0、轴 1 的和与均值，并写出加`keepdims=True`后的形状。

4.6 **广播**。判断`(4, 3)`与`(3,)`、`(4, 1)`与`(3,)`、`(4, 3)`与`(4,)`、`(2, 1, 3)`与`(5, 3)`能否广播并给出输出形状。

4.7 **静默错误**。重算例题 4.3 的差矩阵与 MSE。若标签为 0、0、0，这一错误为何不容易被发现？怎样设计形状检查？

4.8 **矩阵乘法**。令 $A = \left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，$w = (1, 0, -1)$，求`A @ w`、`A * w`及其形状。若`w`改为`(3, 1)`，第一项结果怎样变化？

4.9 **公式到代码**。设 $X : (n, d)$，$W : (d, k)$，$b : (k,)$，写出每样本每输出的预测公式与代码；为什么不能任意把`X @ W`换成`X * W`？

4.10 **随机性**。同一生成器连续调用两次`normal`与两次分别新建同种子生成器有什么区别？为特征和标签分别生成随机排列为什么危险？

4.11 **数值稳定**。对分数 1000、1000、1000 写稳定归一化，解释分母为何不会因全部指数下溢成为 0。输入含 NaN 时能否沿用结论？

4.12 **距离手算**。完成例题 4.6，并写出广播中间形状、归约轴和最近下标。若训练标签为 0、4、8，两个查询的单近邻预测是什么？

4.13 **标准化**。训练第一特征为 0、2、4，第二特征均为 1。按分母 3 求均值与尺度，写出训练标准化结果；预测样本 $(6, 1)$ 应该用哪组参数？

4.14 **复杂度**。查询 1000 条、训练 2000 条、每条 50 特征，直接差数组有多少元素？`float64`仅数据约占多少字节？批大小 20 后最大差数组多大？

4.15 **编程**。实现按查询分块的近邻预测，检查批大小与空输入；批大小 1、2、10 应与完整批量预测一致。说明分块降低了什么成本、没降低什么量级。

4.16 **综合实验**。运行实验 4.2 及答案核对。记录自己环境中的最大数值差与耗时；说明为什么速度不同不必然意味着代码错误，并列出保证算法比较公平的三个条件。

4.16 习题参考答案

4.1–4.5：结构、类型与归约

4.1 `shape`是各轴长度的元组，`ndim`是轴数，`size`是元素总数，`dtype`是元素类型。样本数组形状`(200, 10)`，轴数 2，特征空间维数 10，元素数 2000。该约定需要同时说明行对应样本、列对应特征。

4.2 依次为`(4,)`、`(3,)`、`(3, 1)`、`(1, 4)`、`(4, 3)`。`a.T`第一行为 0、4、8；`a.reshape(4, 3)`第一行为 0、1、2。转置交换轴，重排按指定遍历顺序重新组织；它们不因输出形状相同而相等。

4.3 仍为整数`[1, 2]`，1.9 被转换为整数 1。固定宽度整数计算可能先溢出或截

断，之后转为浮点只能转换已错误的数。正确顺序为先转浮点，再相减或平方，例如`a.astype(np.float64) ** 2`。

4.4 最后a为`[[99, 1, 2], [3, 4, 5]]`。b是基本切片视图，修改传到原数据；c来自高级索引副本，修改不影响a。`b is a`为假仍可能共享内存，因此身份比较不能替代`np.shares_memory`。

4.5 轴 0 和为 5、7、9，均值 2.5、3.5、4.5；轴 1 和为 6、15，均值 2、5。默认形状分别为`(3,)`与`(2,)`；保留轴后分别为`(1, 3)`与`(2, 1)`。归约消去的是指定轴，保留轴只改变形状而非数值。

4.6–4.9：广播与乘法

4.6 四组依次为兼容`(4, 3)`、兼容`(4, 3)`、不兼容、兼容`(2, 5, 3)`。第三组最右轴 3 与 4 冲突；第四组将后者补成`(1, 5, 3)`，前两轴分别按 2 与 1、1 与 5 扩展。

4.7 差矩阵为例题中的 3 行 3 列，平方和 12，平均 4/3。全零标签时所有两两差都为 0，错误结果也为 0，因此只用常数标签测试不足。接口先检查`y.ndim == 1`、`p.shape == y.shape`和非空有限性，再计算；不通过时拒绝，而非自动广播。

4.8 `A @ w`为`[-2, -2]`，形状`(2,)`；`A * w`为`[[1, 0, -3], [4, 0, -6]]`，形状`(2, 3)`。权重改成`(3, 1)`后，矩阵乘法结果为`[[-2], [-2]]`，形状`(2, 1)`。数值排列方向变化会影响后续标签配对，不能只看两个 -2。

4.9 $\widehat{Y}_{i\ell} = \sum_j X_{ij} W_{j\ell} + b_\ell$，代码`Y_hat = X @ W + b`，结果`(n, k)`。逐元素乘法没有沿 j 求和，而且一般`(n, d)`与`(d, k)`本身无法广播；即使某些轴长巧合，意义仍不同。

4.10–4.14：随机、稳定性与规模

4.10 同一生成器两次调用推进状态，取得连续但通常不同的数；两次新建相同种子生成器会从各自序列起点开始，在固定条件下取得相同抽样。特征与标签必须使用同一个排列；分开生成两次排列通常破坏原配对。还需保存原数据顺序、生成器版本与最终划分标识。

4.11 减最大值 1000 后为 0、0、0，指数为 1、1、1，结果各为 1/3。一般有限分数中至少一个最大项减去自身为 0，故至少一项指数为 1，分母至少为 1。含 NaN 时最大值与减法可能传播 NaN，有限性假设已被破坏，应先拒绝或按明确的数据规则处理。

4.12 差数组形状`(2, 3, 2)`，沿轴 2 求平方和得到`[[1, 1, 5], [9, 13, 1]]`。沿轴 1 取最小位置得到 0、2，同分取靠前记录，预测 0、8。用`argmin(axis=0)`会回答另一个问题，得到每个训练点最接近的查询编号。

4.13 均值为 $(2, 1)$，第一列方差 $(4 + 0 + 4)/3 = 8/3$，标准差 $\sqrt{8/3}$，第二列尺度按规则设为 1。标准化后第一列为 $(-\sqrt{3/2}, 0, \sqrt{3/2})$，第二列全 0。新样本仍用训练均值与尺度，得到 $(4/\sqrt{8/3}, 0) = (\sqrt{6}, 0)$，不能给单样本另估均值使其变为 0。

4.14 元素数为 $1000 \times 2000 \times 50 = 10^8$，每元素 8 字节，共 8×10^8 字节，约 0.8 GB，不含平方中间量等额外占用。批大小 20 时为 $20 \times 2000 \times 50 = 2 \times 10^6$ 元素，约

1600 万字节，即 16 MB。这里使用十进制 GB 与 MB，二进制单位换算数值不同。

4.15–4.16：编程与实验

4.15 复用完整的`predict_vectorized`，按查询切片处理。核心答案为：

```
def predict_blocked(train, target, query, k, block_size=2):
    if (not isinstance(block_size, int) or isinstance(block_size, bool)
            or block_size < 1):
        raise ValueError("block_size must be positive")
    if len(query) == 0:
        raise ValueError("empty query")
    blocks = []
    for start in range(0, len(query), block_size):
        part = query[start:start + block_size]
        blocks.append(predict_vectorized(train, target, part, k))
    return np.concatenate(blocks)
```

本答案接口假定`query`已经是数组；组内函数继续验证二维、有限、特征数与标签。批大小与 k 一样拒绝布尔值，不把`True`当成 1。答案脚本用训练点 $(0,1),(2,1),(4,1)$、标签 0、4、8 与查询 $(1,1),(3,1),(5,1)$，同分取靠前，预测 0、4、8；三个批大小应完全一致。分块降低峰值中间内存，不降低全部距离计算的 $O(mnd)$ 量级。

4.16 正确结果先满足形状一致、无非有限数以及指定容差，再比较耗时。环境中的数值库、线程数、硬件、后台负载与计时噪声都可能改变速度。公平比较应使用相同数值输入与类型、同一预测规则和同分约定、相同计时范围；先预运行、重复测量，并排除读取和打印。若改了预处理或 k，就已在比较不同算法条件，需重新解释评价协议。

4.17 本章小结

NumPy 把数值内容、类型和形状放在同一数组结构中。逐元素运算描述位置上的变换，归约消去指定轴，广播在兼容轴间复用输入，矩阵乘法执行内侧下标上的乘积求和。理解每个轴的意义，是判断这些规则是否实现正确计算的前提。

向量化提高执行效率的同时，也可能建立很大的中间结果。可靠的数值程序还要管理共享内存、随机状态和浮点误差；完整实验则继续遵守训练统计量只从训练数据估计、验证负责选择、测试负责最终评价的流程。